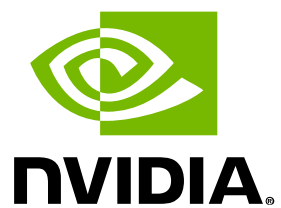


LECTURE 1: MOTIVATION AND BASIC CONCEPTS OF LARGE MODEL TRAINING



DEEP
LEARNING
INSTITUTE

COURSE AGENDA

08:50	—	09:00	Welcome
09:00	—	10:00	Lecture 1: Motivation and basic concepts
10:00	—	11:00	Lab 1 /Part 1: From SLURM basics to GPT model pretraining
11:00	—	11:15	Break
11:15	—	12:15	Lecture 2: Advanced concepts of large model training
12:15	—	13:15	Lab 1 /Part 2: Multi-node scaling
13:15	—	14:00	Lunch break
14:00	—	15:00	Lecture 3: Large model deployment
15:00	—	16:00	Lab 2: Hands on example of how to deploy LLama2 into Triton Inference Server
16:00	—	17:00	Assessment support

PART 1

Motivation and basic concepts

- Lecture
 - **Why large models?**
 - Impact on AI landscape
 - Challenges of large model training
 - Basic techniques for memory reduction
 - Overview of the tools used in the lab
- Lab 1 / Part 1
 - Introduction to the SLURM class environment
 - GPT model pretraining
 - Multi-node scaling
 - Optimize the GPT model pretraining

DRAMATIC INCREASE IN MODEL SIZES

The Trend Continues

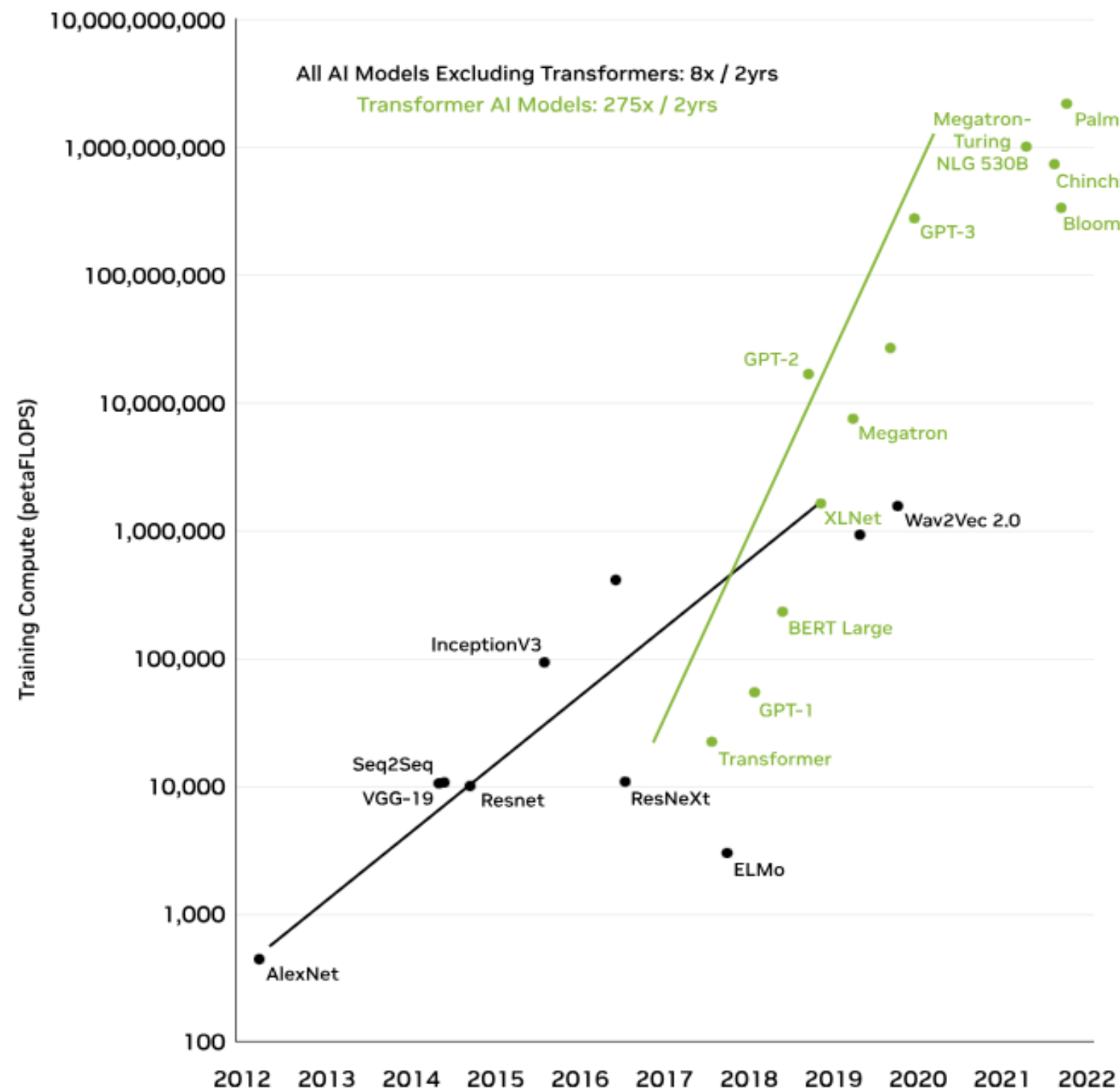


Figure 3. Compute required for training transformer models.

- 2017 paper - Attention is All you Need introduced the transformer architecture
 - Removed the sequential processing dependency of RNNs
 - Enabled language models to be trained with parallelism
 - Beginning of the Large Model Revolution
- BERT from Google in 2019
 - Trained on large corpus of text data and applied embeddings to other tasks
 - Popularized finetuning of models to get task specific embeddings
- Megatron-LM (NVIDIA) 2021 - showed how to scale training efficiently to thousands of GPUs with model parallelism which led to the explosion of larger models
- GPT-3 was the precursor to the popular chat-gpt which made LLMs commercially viable

TRANSFORMING SPEECH PROCESSING

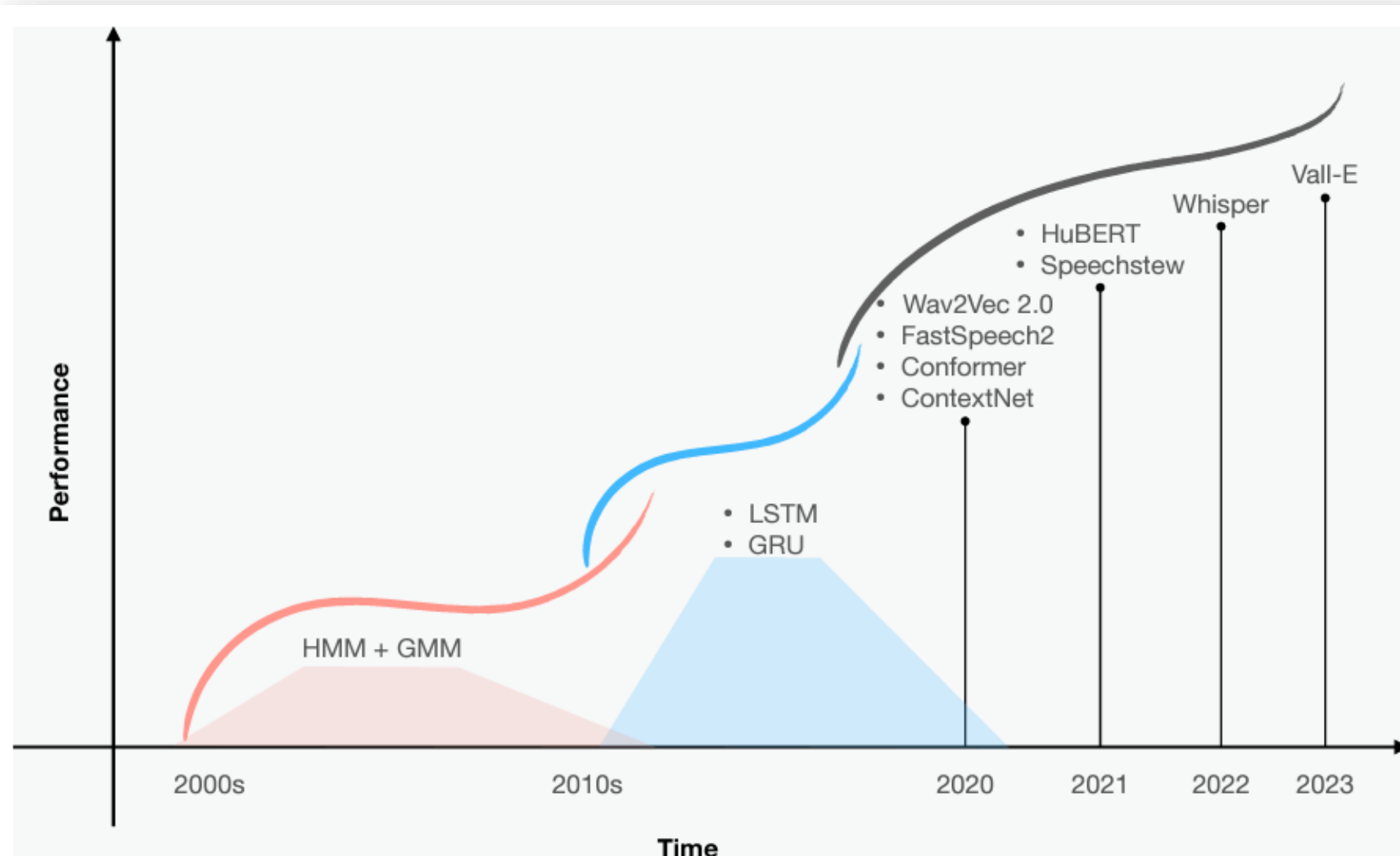


Fig. 1. Evolution of speech processing models over the years.

Speech processing accuracy has improved significantly over time

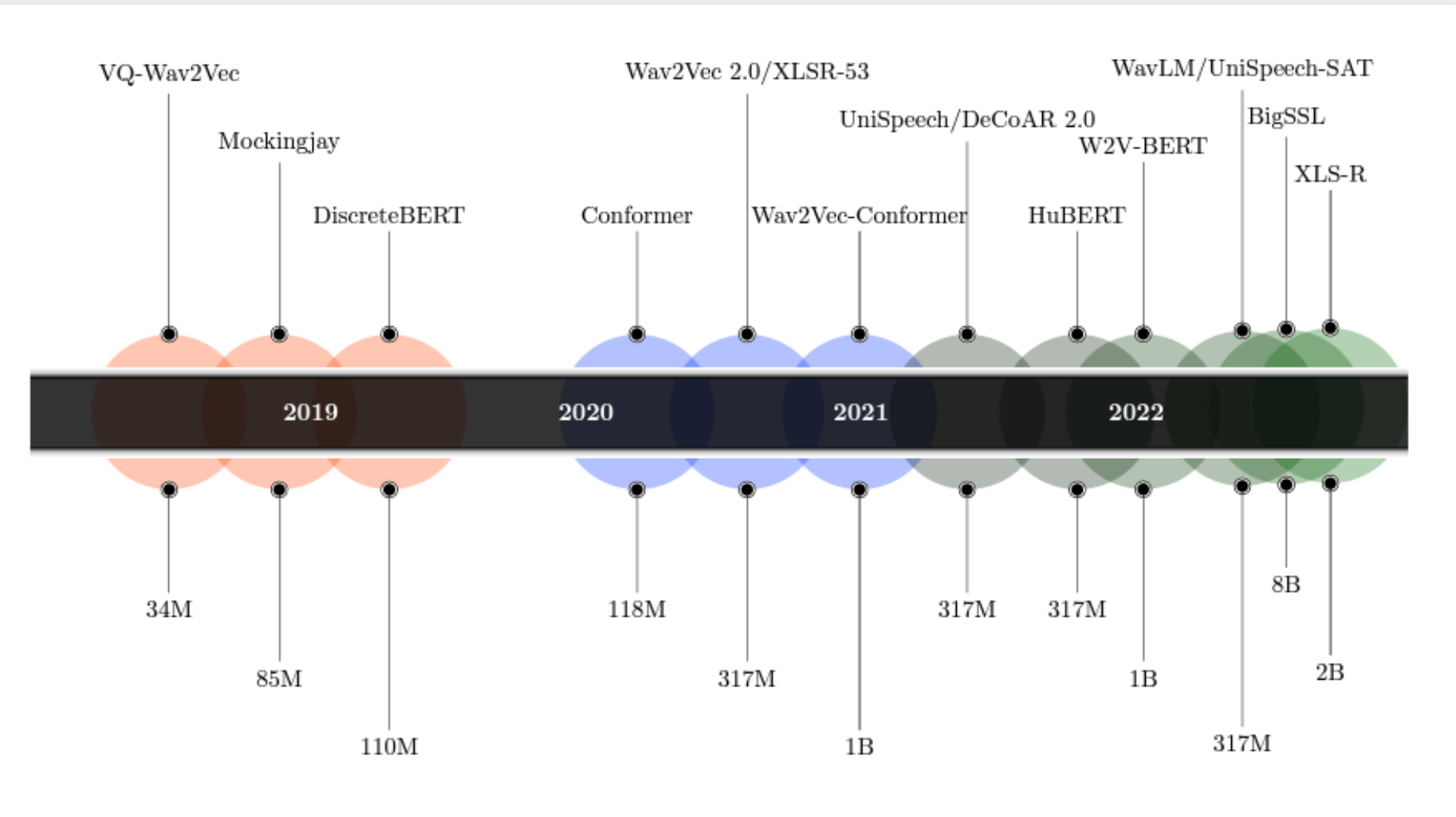


Fig. 4. Timeline highlighting notable large Transformer models developed for speech processing, along with their corresponding parameter sizes.

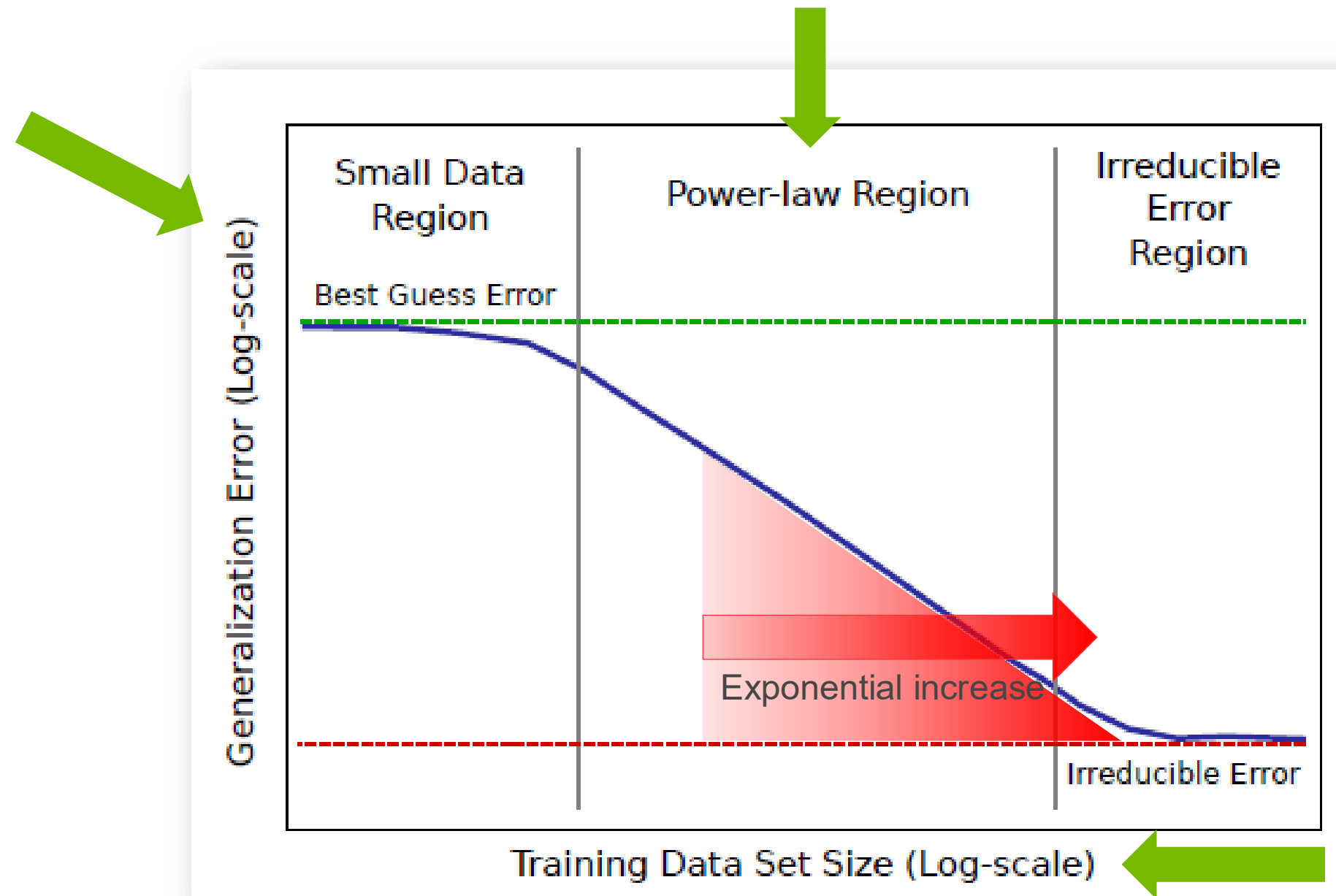
Increase in performance of model tracks with increase in model size

REASONS OF SUCCESS OF LARGE NEURAL NETWORKS



THE COST OF LABELS

Limits the utility of supervised deep learning models



To reduce generalization error, significant amounts of costly labelled training data is needed

THE COST OF LABELS

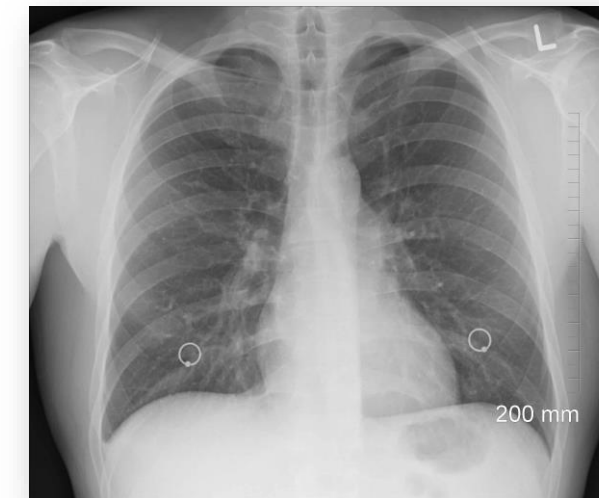
Hard limits to supervised deep learning utility

Self-Driving Auto Example



Thousands of human data labellers are needed to create effective training data

Clinical Radiology Example



Labelling medical image data requires highly skilled radiologists

Table 1. UK clinical radiology workforce (headcount), 2018

	England	Northern Ireland	Scotland	Wales	UK total
Consultant-grade	3,296	135	327	169	3,927
Trainees	1,286	51	149	69	1,555
SAS-grade*	65	0	2	2	69
Total	4,647	186	478	240	5,551

[SAS grade comprises associate specialists, specialty doctors and trust-grade staff.]

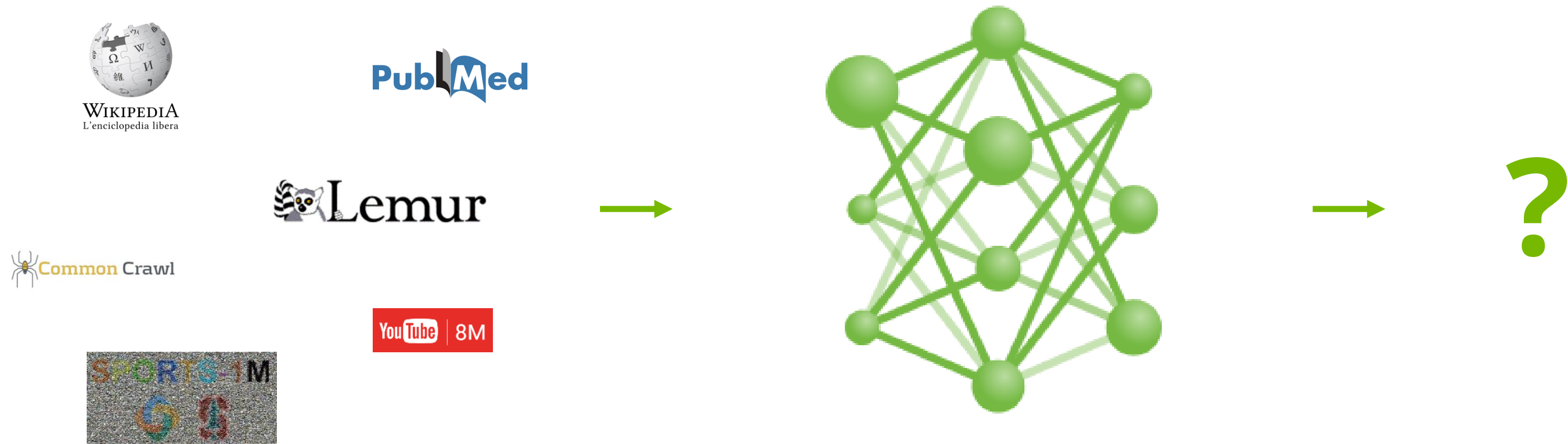
Not enough available skilled people to label training data, only 3296 radiologists in England

SELF-SUPERVISED LEARNING

The background of the slide features a smooth gradient from a deep green on the left to a bright yellow on the right. Overlaid on this gradient is a complex, abstract network of white dots and thin white lines, resembling a neural network or a data visualization. The network is denser on the right side, where it also appears to form a faint, wireframe-like structure of a face.

SELF AND UN-SUPERVISED LEARNING

What to do in the absence of labels



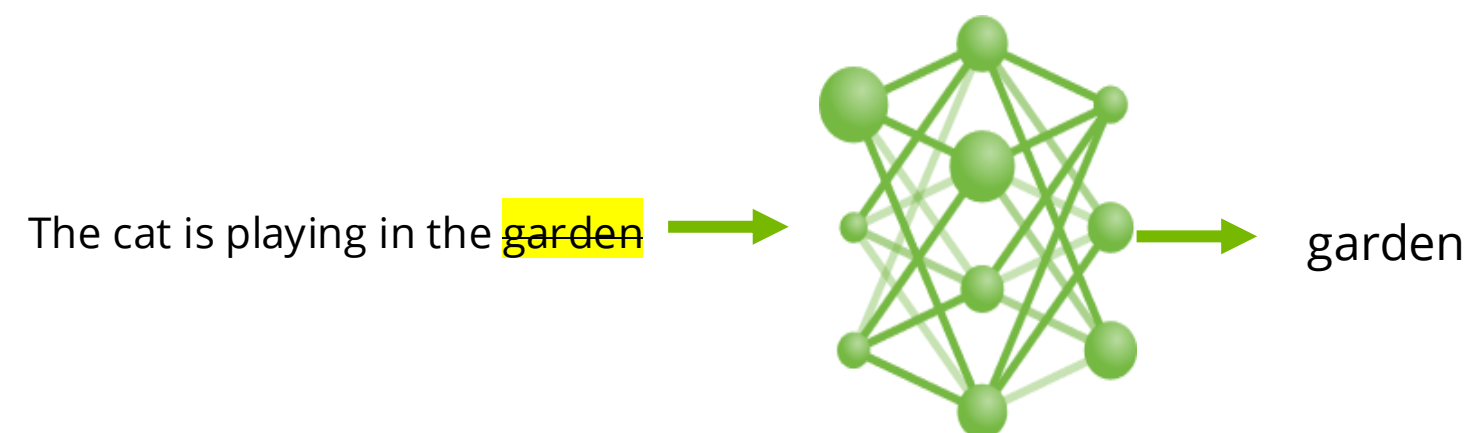
Is there a way to use publicly available (unlabelled) text data to train a large language model to do next token prediction?

SELF AND UN-SUPERVISED LEARNING

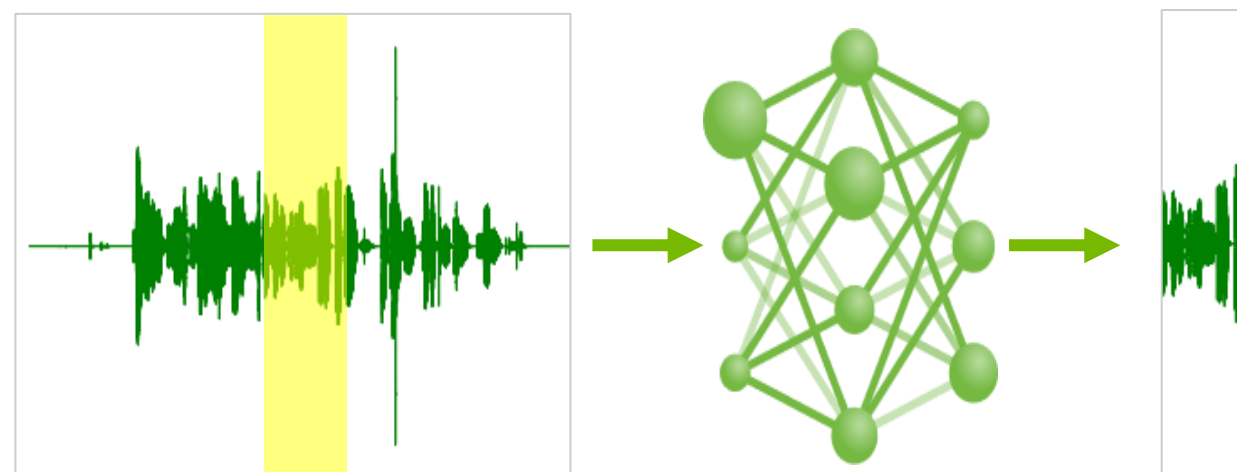
What to do in the absence of labels

Masking Loss

Omit data and force the model to predict what missing data was



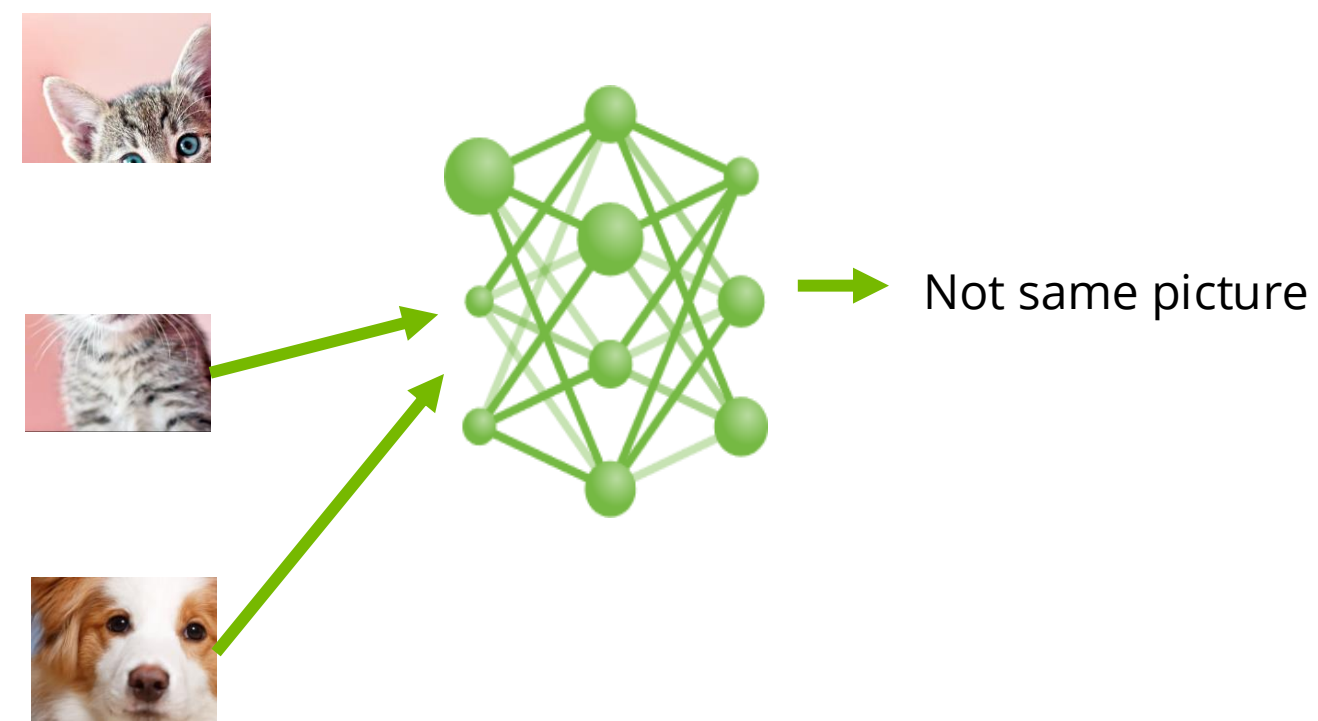
Masking loss applied to text prediction



Masking loss applied to Speech Processing

Contrastive Loss

Use out of distribution data to condition model on in-distribution data

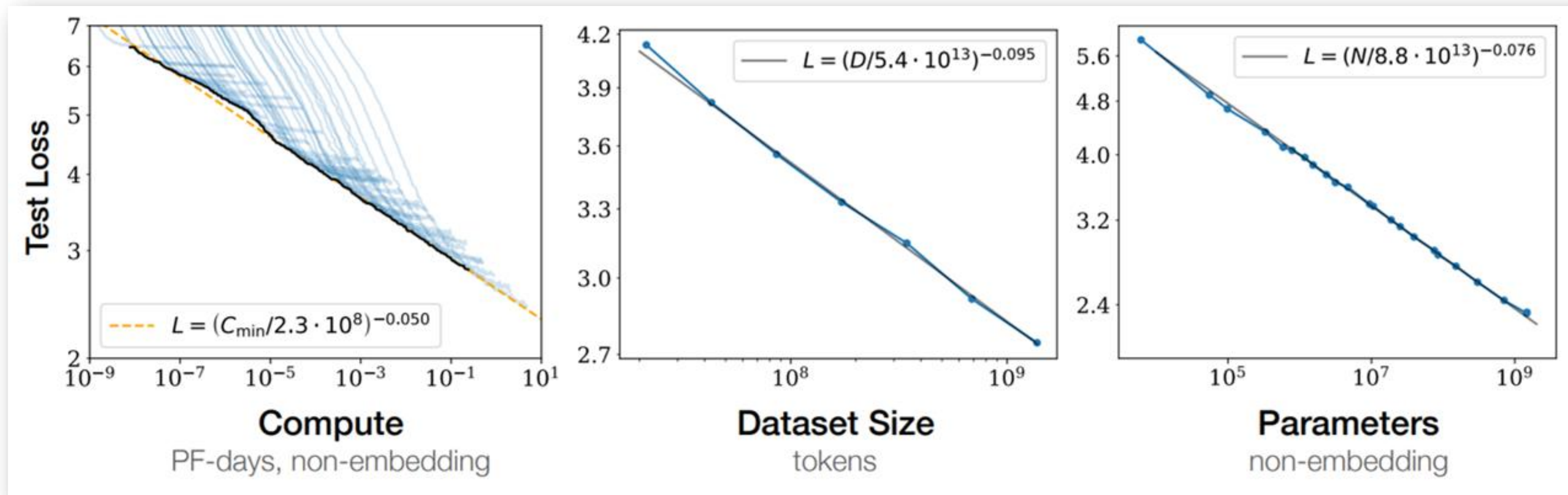


THE SCALING LAWS



EMPIRICAL EVIDENCE

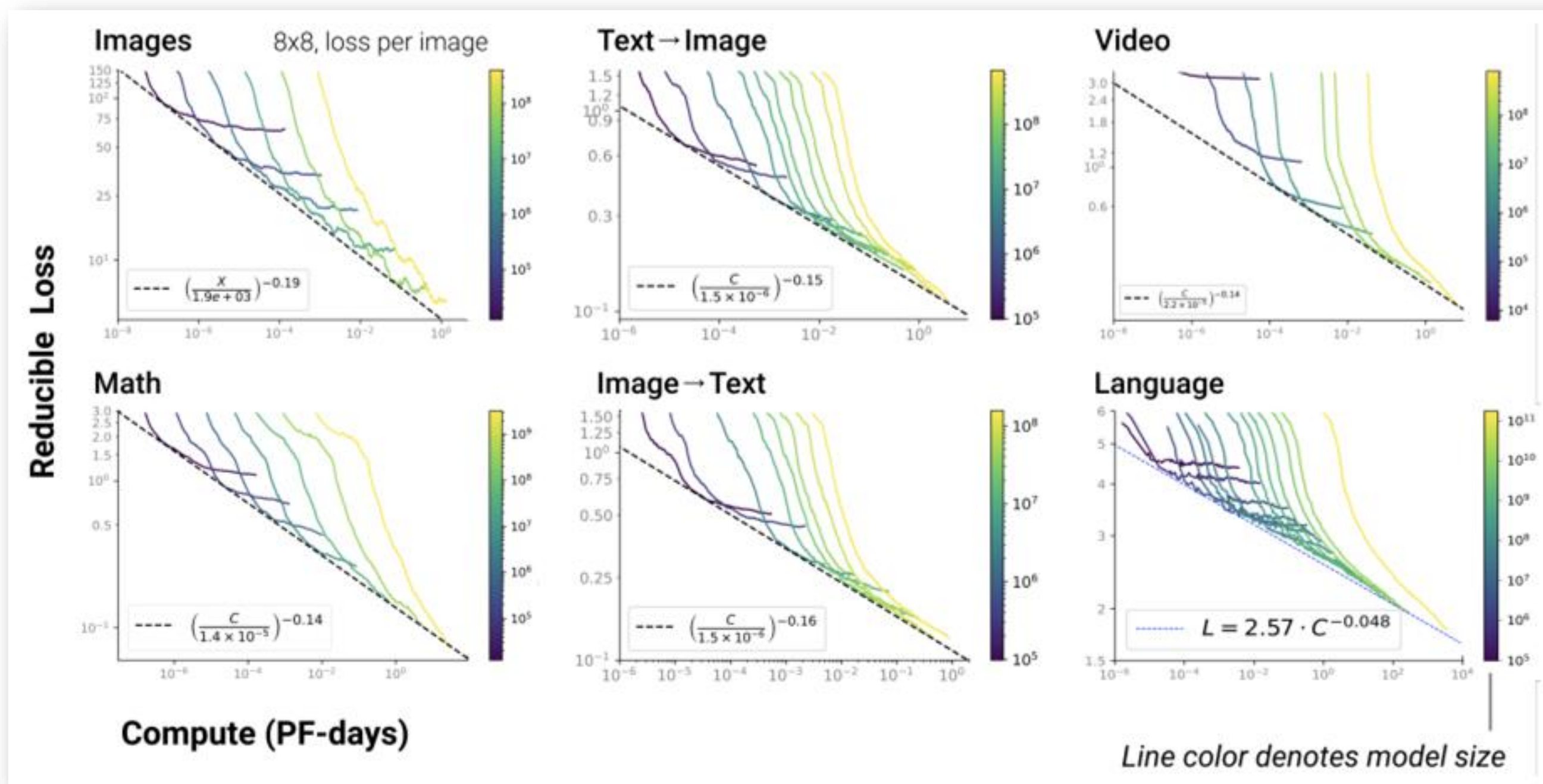
The Scaling Laws in NLP



Error rate decreases with a **simultaneous** increase in compute, dataset size and model size

EMPIRICAL EVIDENCE

The Scaling Laws for generative models



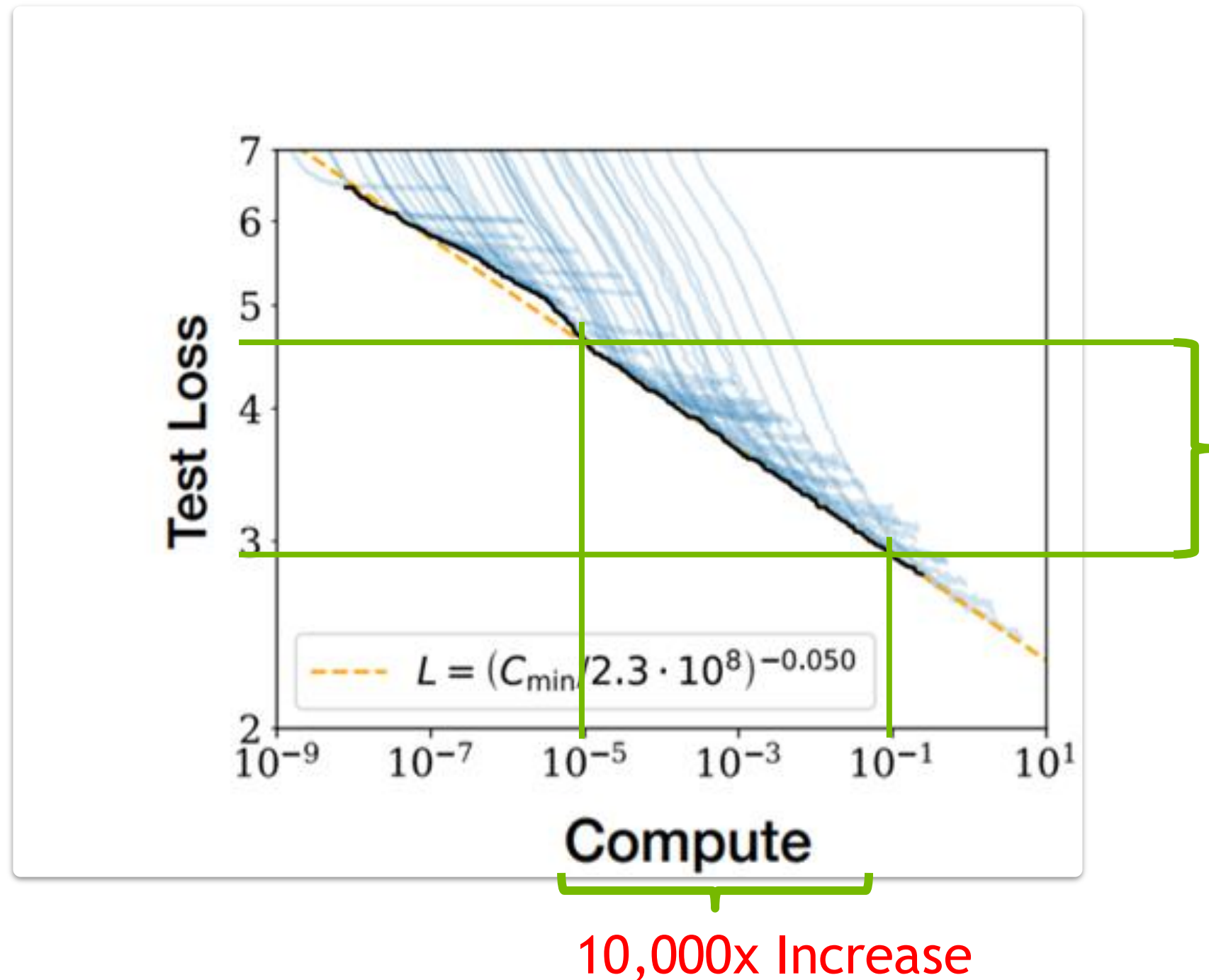
Error rate decreases with a **simultaneous** increase in compute, dataset size and model size

THE COST

The background of the slide features a smooth gradient from a deep blue at the top to a vibrant green at the bottom. Overlaid on this gradient is a complex, white, network-like pattern. This pattern consists of numerous small dots connected by thin, intersecting lines, creating a mesh or web-like structure that is denser on the right side and fades towards the left.

ARE LARGE LANGUAGE MODELS WORTH IT?

The cost of incremental improvement



- Plots shows test set loss on y-axis and increasing compute scale on the x-axis
- As compute grows you get a decrease in test loss error
- The reduction in test loss error is not directly proportional to the increase in compute
- Why are AI companies interested in using large compute if accuracy doesn't improve that much?

BEYOND ACCURACY

The background features a smooth gradient from a vibrant green on the left to a deep blue on the right. Overlaid on this gradient is a complex, abstract network of white dots and thin white lines, resembling a molecular structure or a data network. The network is denser on the right side of the image, where it also forms a faint, large-scale grid pattern.

FEW SHOT LEARNING

Learning from far fewer examples

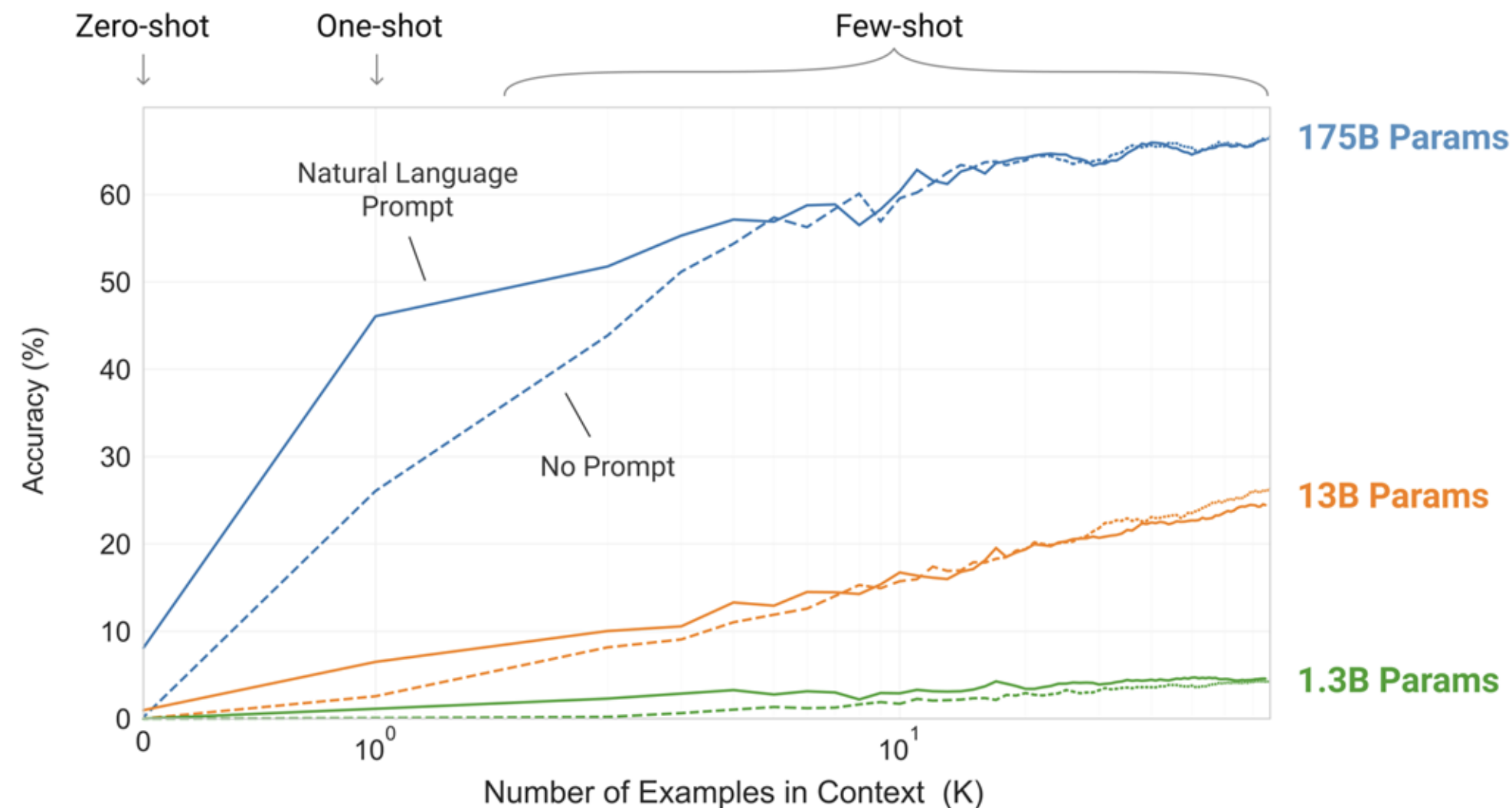


Figure 1.2: Larger models make increasingly efficient use of in-context information. We show in-context learning performance on a simple task requiring the model to remove random symbols from a word, both with and without a natural language task description (see Sec. 3.9.2). The steeper “in-context learning curves” for large models demonstrate improved ability to learn a task from contextual information. We see qualitatively similar behavior across a wide range of tasks.

Large models are more sample efficient needing less epochs to train and fewer examples learn new tasks



EXAMPLE OF FEW SHOT LEARNING (PROMPT DESIGN)

Prompt Megatron

Add prompt

Number tokens to generate (1-1024):

0 / 1000

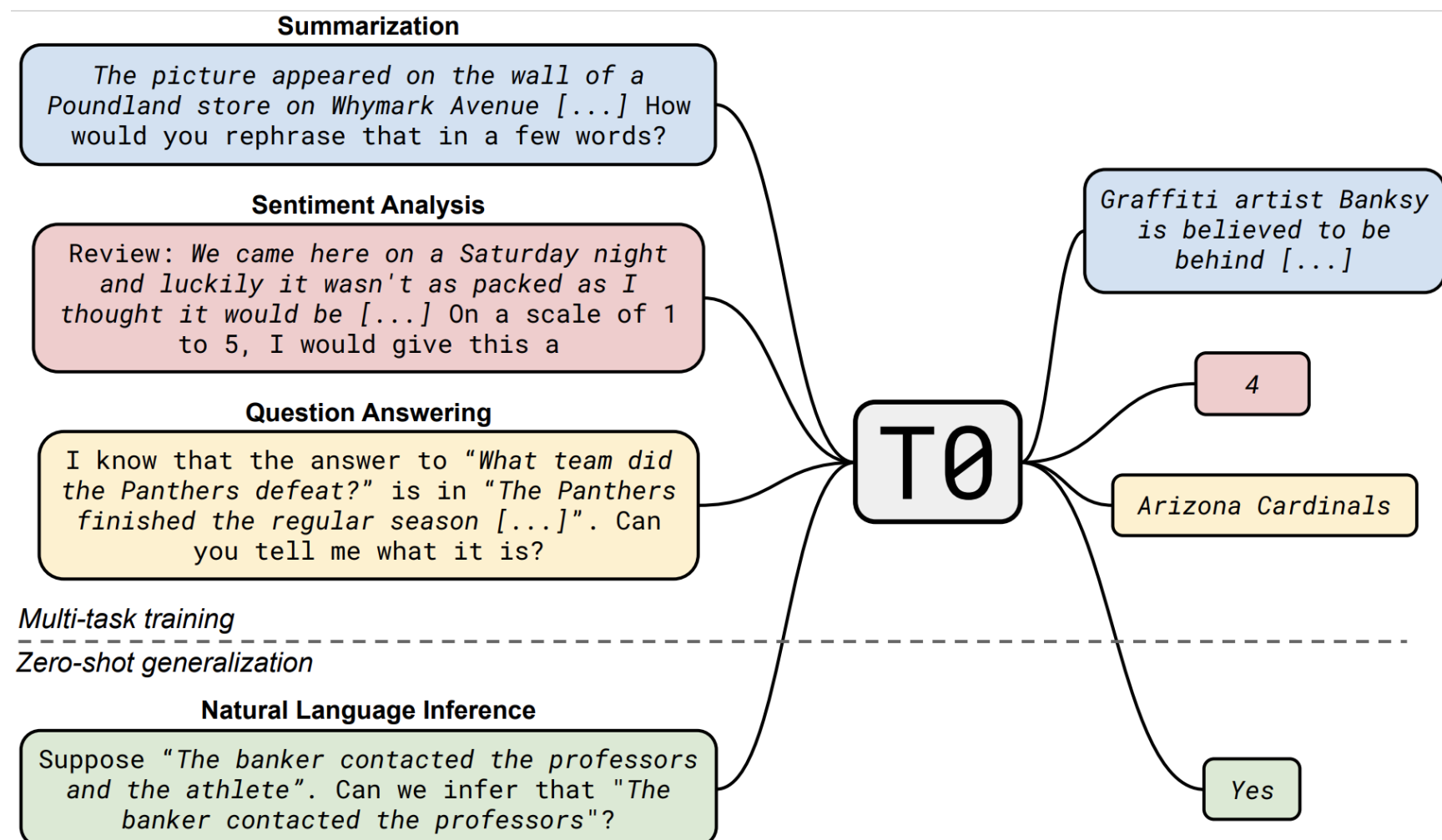
Megatron response...

MULTITASK SKILLS

The background of the slide features a smooth gradient from a deep green on the left to a bright yellow on the right. Overlaid on this gradient is a complex, abstract network of thin white lines connecting numerous small white dots, creating a web-like or molecular structure that spans the entire width of the image.

MULTITASK SKILLS

Generalize to unseen Tasks: T0



- Can you enable better model generalization by training with multiple tasks vs large datasets
- Tasks were specified in natural language prompts curated by volunteers
- Multiple prompts per dataset were captured to improve word choice robustness of prompts
- Trained a T5 model on a subset of tasks and evaluated on tasks not trained on.
- Model was 16x smaller than GPT-3 and exceeded 9/11 hold out sets for performance

Multitask Prompted Training Enables Zero-Shot Task Generalization

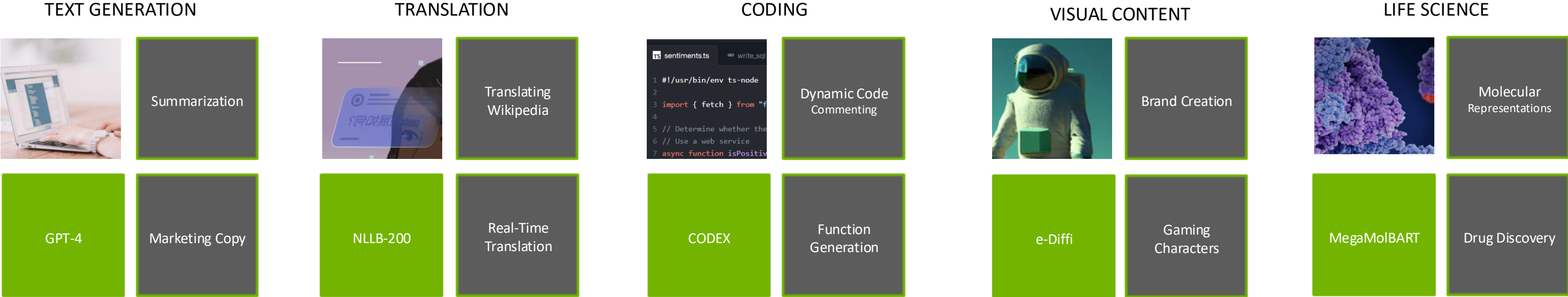
Victor Sanh, Albert Webson, Colin Raffel, Stephen H. Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, Manan Dey, M Saiful Bari, Canwen Xu, Urmish Thakker, Shanya Sharma Sharma, Eliza Szczechla, Taewoon Kim, Gunjan Chhablani, Nihal Nayak, Debajyoti Datta, Jonathan Chang, Mike Tian-Jian Jiang, Han Wang, Matteo Manica, Sheng Shen, Zheng Xin Yong, Harshit Pandey, Rachel Bawden, Thomas Wang, Trishala Neeraj, Jos Rozen, Abheesht Sharma, Andrea Santilli, Thibault Fevry, Jason Alan Fries, Ryan Teehan, Tali Bers, Stella Biderman, Leo Gao, Thomas Wolf, Alexander M. Rush

PART 1

Motivation and basic concepts

- **Lecture**
 - Why large models?
 - **Impact on AI landscape**
 - Challenges of large model training
 - Basic techniques for memory reduction
 - Overview of the tools used in the lab
- **Lab 1 /Part 1**
 - Introduction to the SLURM class environment
 - GPT model pretraining
 - Multi-node scaling
 - Optimize the GPT model pretraining

GENERATIVE AI IS TRANSFORMING BUSINESS

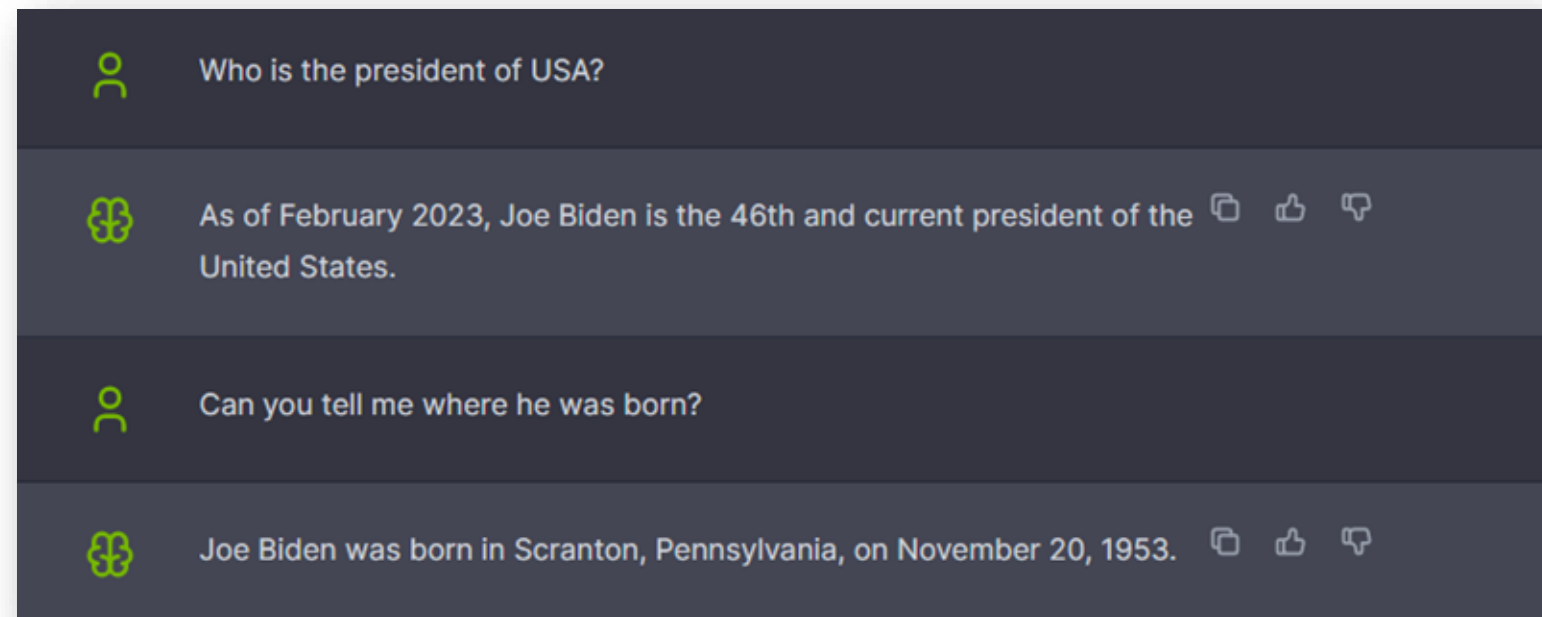


Enterprises that adopt next-generation AI like LLMs and Generative AI are **2.6X more likely to increase revenue by 10% or more** but must invest in their AI infrastructure to fully reap the benefits.

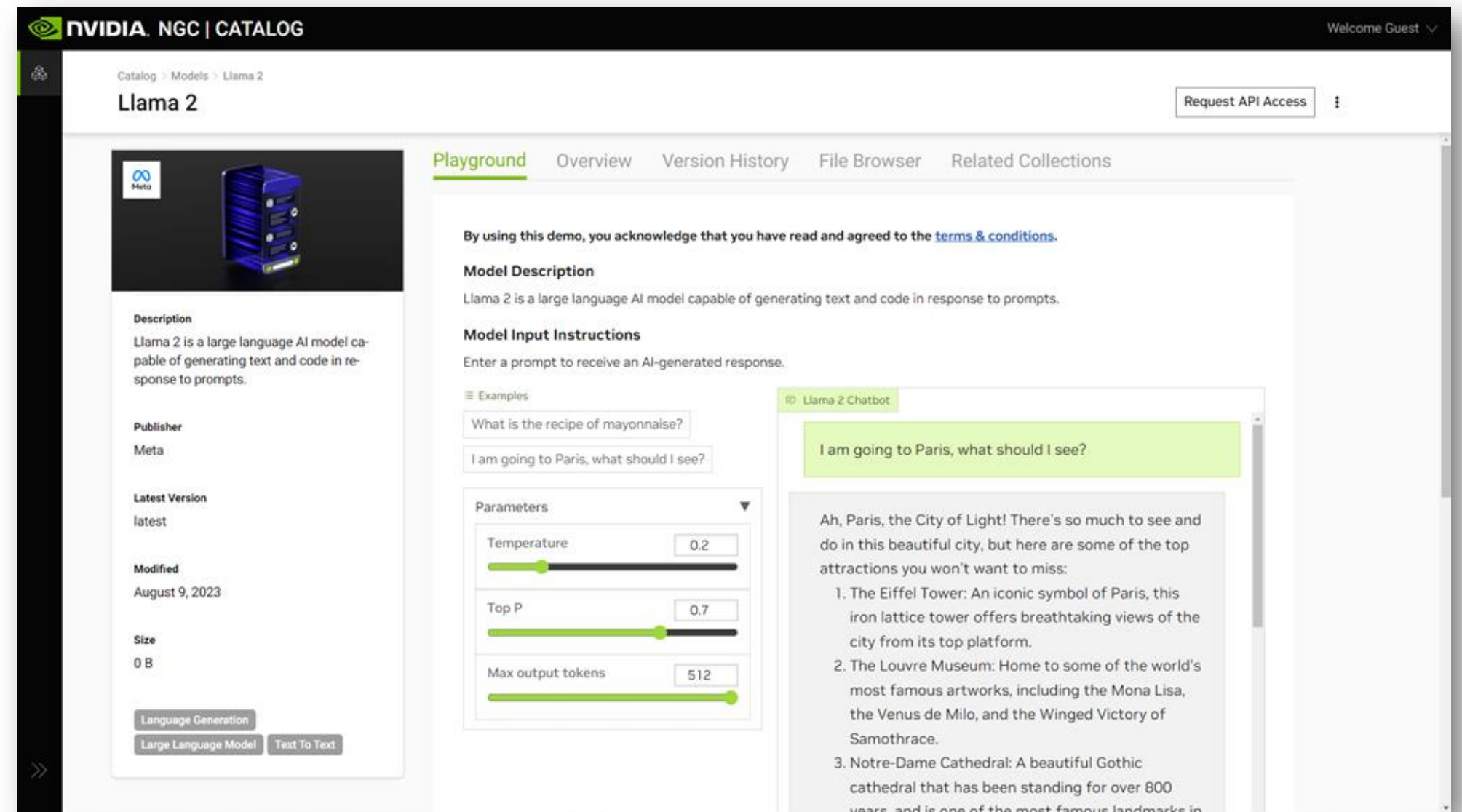
-Accenture Research. Breakthrough Innovation: Is your organization equipped for breakthrough innovation? WEF 2023.

USE CASES FOR LARGE MODELS

Chatbots | Virtual Assistants



Example of multi-turn chatbot using LLMs



[Llama2 Chat on NVIDIA NGC Playground](#)

Free hosted LLMs available for public on ngc.nvidia.com

USE CASES FOR LARGE MODELS

Code Generation

A: Can you express that in JavaScript?

Certainly! Here's an example of how the algorithm for synthesizing dimethyl carbonate (DMC) from raw materials could be expressed in JavaScript:

```
javascript
// Set initial conditions
let methanol = 2; // Molar ratio of methanol to phosgene
let phosgene = 1;
let catalyst = 'quaternary ammonium salt'; // or 'tertiary amine'
let temp = 60; // Temperature in degrees Celsius
let reaction_time = 60; // Reaction time in minutes
let DMC;

// Run the reaction
function run_reaction() {
  // Add catalyst to the mixture
  let mixture = `${methanol} moles of methanol and ${phosgene} moles of phosgene`;

  // Heat the mixture to the desired temperature and maintain for the desired time
  let reaction = `Heat the mixture to ${temp} °C and maintain for ${reaction_time} minutes`;

  // Collect and condense the DMC vapor
  DMC = `Collect the DMC vapor produced and condense it into a liquid`;

  // Purify the DMC liquid
  DMC = `Purify the DMC liquid by distillation or other techniques to remove impurities`;

  // Call the function to run the reaction
  run_reaction();
}
```

Of course, this is a simplified example for illustrative purposes only, and should not be used for actual chemical synthesis without proper safety measures and equipment.

ChatGPT

NVIDIA NGC | CATALOG

Catalog > Models > Code Llama

Code Llama

Request API Access

Playground Overview Version History File Browser Related Collections

By using this demo, you acknowledge that you have read and agreed to the [terms & conditions](#).

Model Description
CodeLlama is a large language AI model capable of generating text and code in response to prompts.

Model Input Instructions
Enter a prompt to receive an AI-generated response.

Examples

- Write a function that computes the set of sums of all contiguous sublists of a given list.
- Write the Fibonacci sequence in Python.

Parameters

- Temperature: 0.2
- Top P: 0.7
- Max output tokens: 512

Code Llama Chatbot

To read a CSV file in Python, you can use the csv module. Here's an example of how to do it:

```
import csv

# Open the CSV file
with open('data.csv', 'r') as csvfile:
    # Create a CSV reader object
    reader = csv.reader(csvfile)

    # Read the header row
    header = next(reader)

    # Read the data rows
    for row in reader:
        # Process the data row
        print(row)
```

This code opens the CSV file, creates a CSV reader object,

Code Llama Chatbot NVIDIA NGC Playground

	Easy		Median		Hard		Overall	
pass@k	k = 1	k = 5	k = 1	k = 5	k = 1	k = 5	k = 1	k = 5
GPT-4	68.2	86.4	40.0	60.0	10.7	14.3	38.0	53.0
text-davinci-003	50.0	81.8	16.0	34.0	0.0	3.6	19.0	36.0
Codex (code-davinci-002)	27.3	50.0	12.0	22.0	3.6	3.6	13.0	23.0
Human (LeetCode users)	72.2		37.7		7.0		38.2	

Table 2: Zero-shot pass@1 and pass@5 accuracies (%) on LeetCode.

Sparks of Artificial General Intelligence: Early experiments with GPT-4

GPT-4 surpasses Zero-shot benchmarks for coding

USE CASES FOR LARGE MODELS

Plan and Execute

GPT-4: tool warmup

The computer is answering questions. If the computer needs any current information to answer the question, it searches the web by saying `SEARCH("query")`, reads the snippets in the result, and then answers the question. If it needs to run any calculations, it says `CALC(expression)`, and then answers the question. If it needs to get a specific character from a string, it calls `CHARACTER(string, index)`.
</endofprompt>

Q: Who is the current president of the United States?

Computer: `SEARCH("current US president")`

> `SEARCH SNIPPETS`

[1] Joe Biden: The President | The White House

President Joe Biden Vice President Kamala Harris First Lady Dr. Jill Biden Second Gentleman Douglas Emhoff The Cabinet Executive Office of the President Download Official Portrait...

[2] Who is the current US president? | The US Sun

Who is the current US president? Katrina Schollenberger Jennifer Roback NY Published: 8:30 ET, Jul 2 2022 Updated: 8:39 ET, Jul 2 2022 DONALD Trump controversially defeated Hillary Clinton in the 2016 Presidential Election, making him the 45th president of the United States.

[3] President of the United States - Wikipedia

Joe Biden is the 46th and current president of the United States, having assumed office on January 20, 2021. Contents 1 History and development 1.1 Origins 1.2 1789{1933 1.3 Imperial Presidency 1.4 Critics of presidency's evolution 2 Legislative powers 2.1 Signing and vetoing bills 2.2 Setting the agenda 2.3 Promulgating regulations

Computer: Joe Biden

Q: What is `sqrt(34324*2432)`?

Computer: `CALC(sqrt(34324*2432))`

> 9136.518374085394

Computer: 9136.518374085394

Q: What is the 13th letter of the word 'supralapsarian'?

Computer: `CHARACTER('supralapsarian', 13)`

> a

Computer: The 13th letter of the word 'supralapsarian' is a.

Figure 5.2: Examples of GPT-4 using various tools to answer questions.

[GPT-4 Technical Report](#)

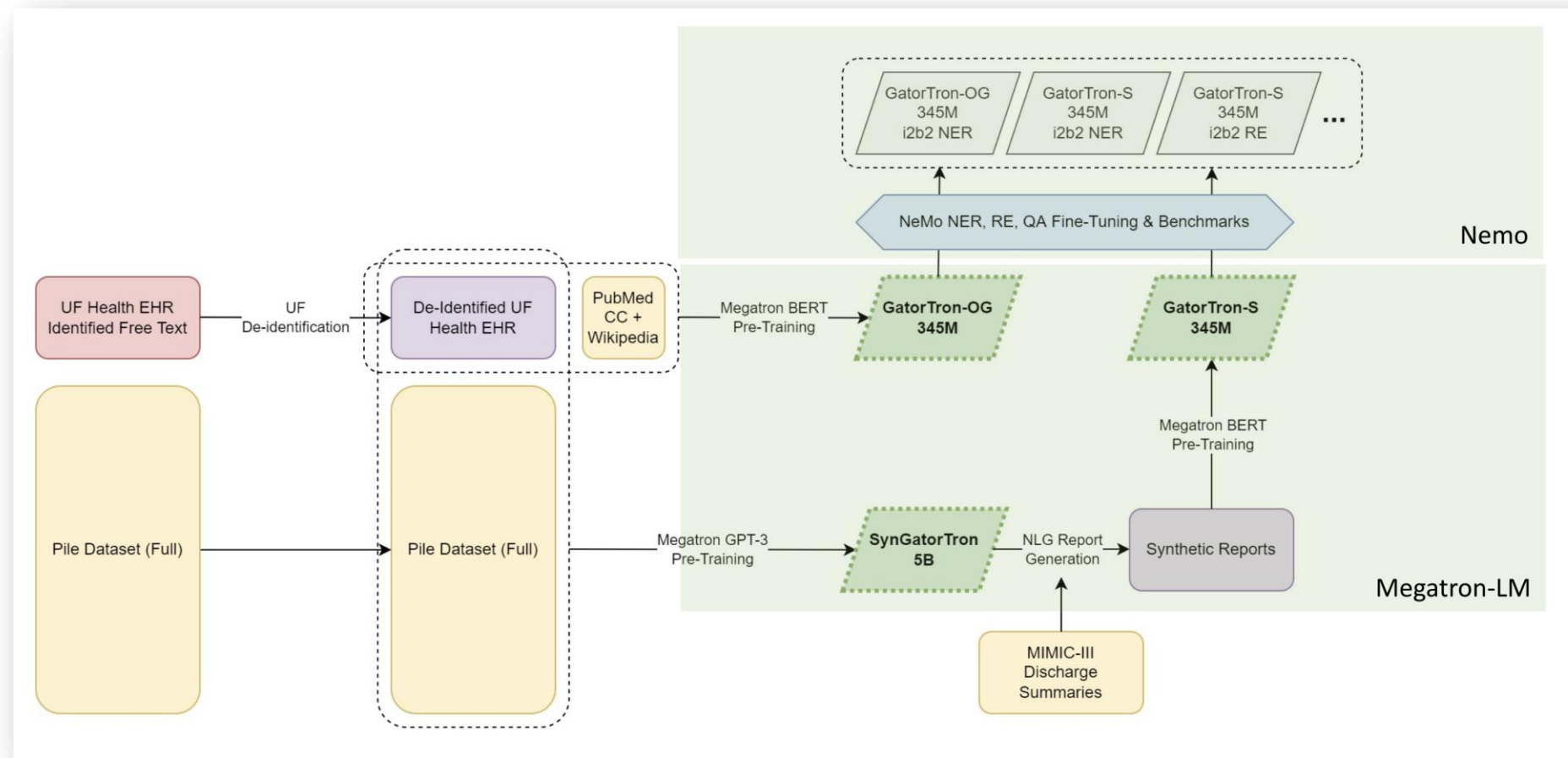


Figure 1: The task-solving pipeline in OpenAGI.

[OpenAGI](#)

USE CASES FOR LARGE MODELS

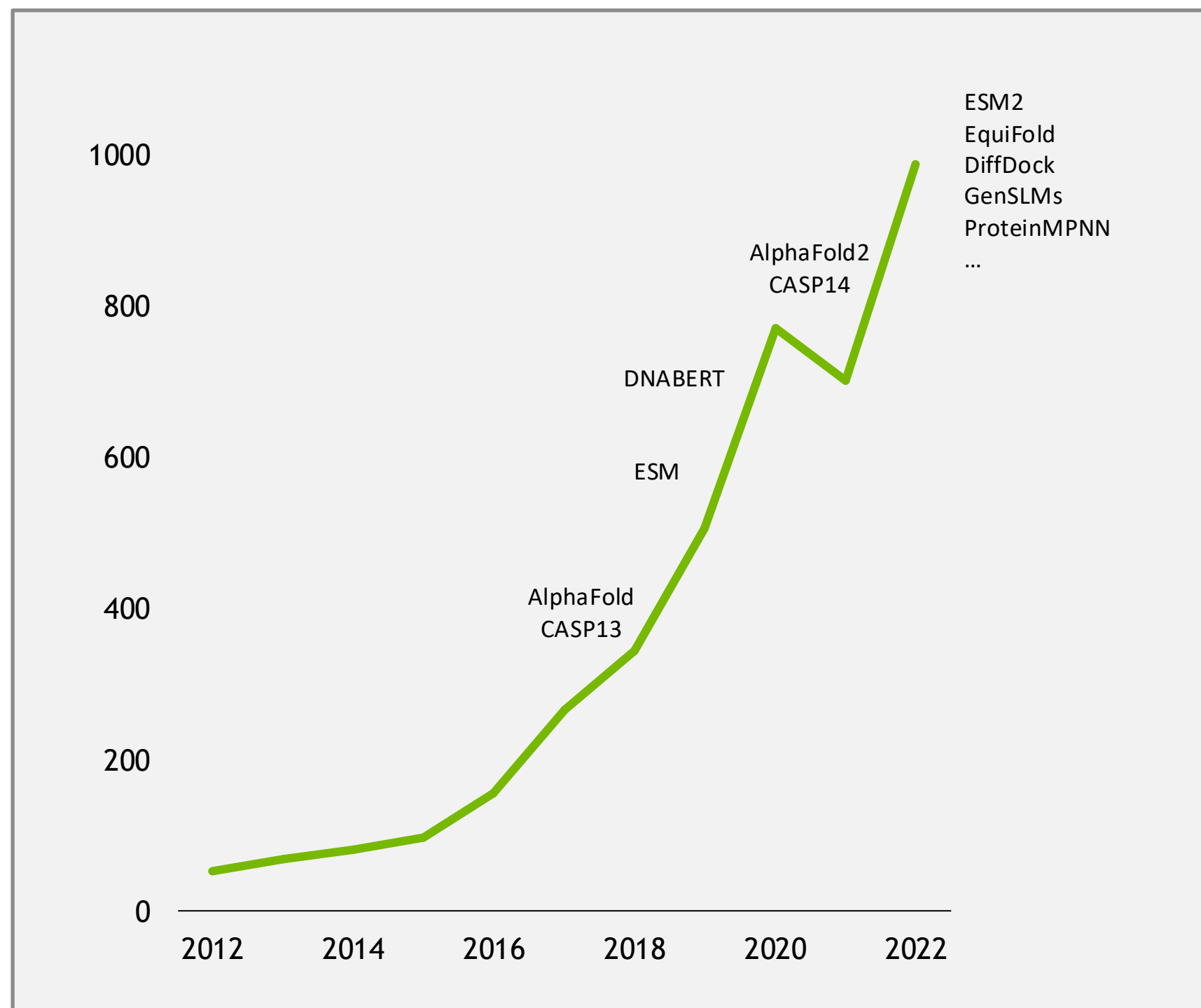
Clinical Language: SynGatorTron



- 5B Language model for creating synthetic patient profiles
- Largest LLM in healthcare so far (2024)
- Researchers use synthetic data to create tools, models without risk of violating privacy
- Developing models on synthetic data allows more collaboration across research groups
- GatorTron-S - bert style model trained on synthetic data from SynGatorTron
- Trained using Megatron-LM library and DGX Superpod

GENERATIVE AI IS EXPLODING IN DRUG DISCOVERY

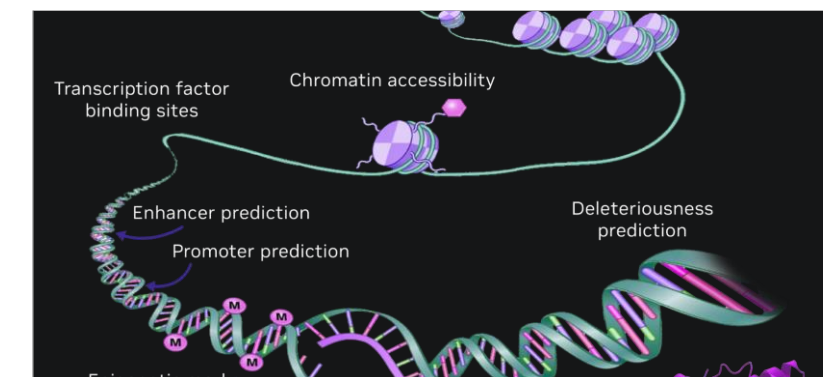
AI Published Papers



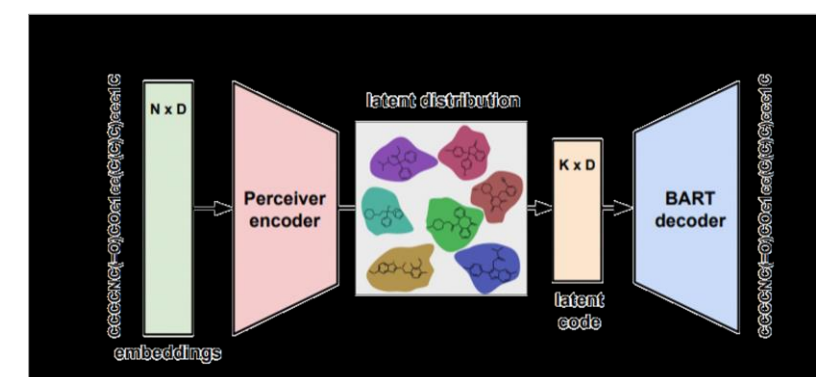
NVIDIA Joint Research



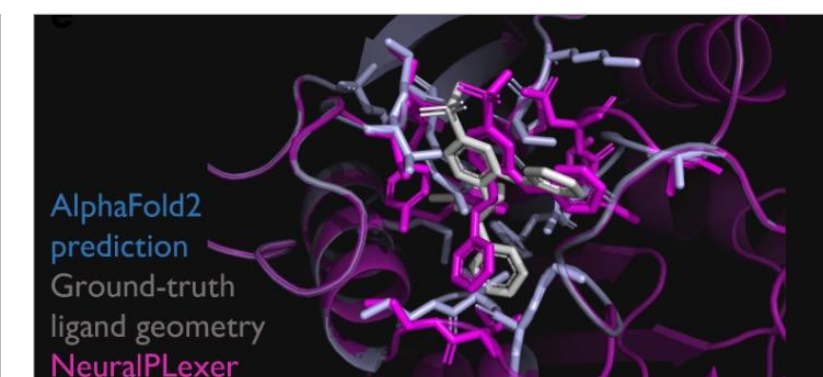
GensLMs
Genome-scale language models reveal SARS-CoV-2
Evolutionary Dynamics



Nucleotide Transformer
Building and Evaluating Robust Foundation Models for
Human Genomics



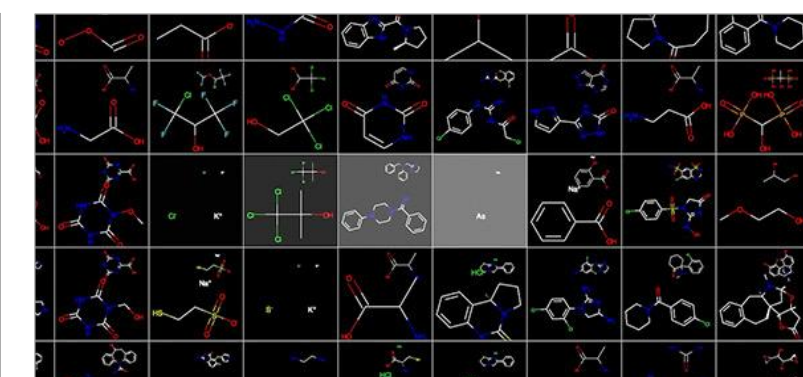
MoIMIM
Improving Small Molecule Generation using
Mutual Information Machine



NeuralPlexer
Dynamic-Backbone Protein-Ligand Structure Prediction with
Multiscale Generative Diffusion Models



ProT-VAE
Protein Transformer Variational AutoEncoder for Functional
Protein Design

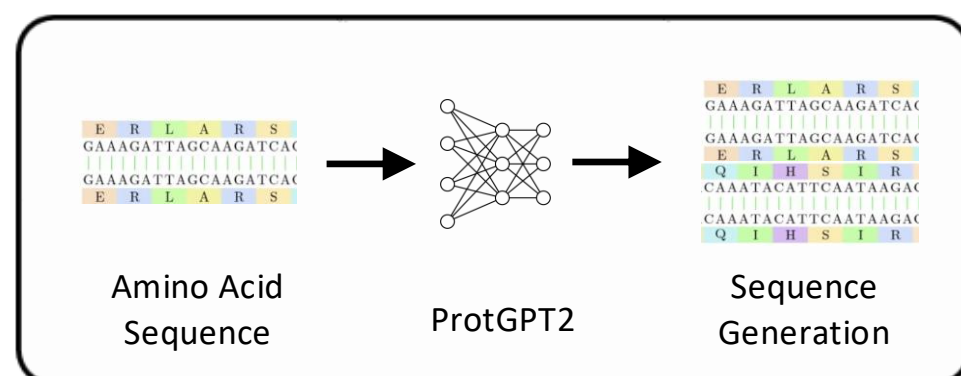


MegaMoIBART
Generally Applicable Chemical AI Models with Large-Scale
Pretrained Transformers

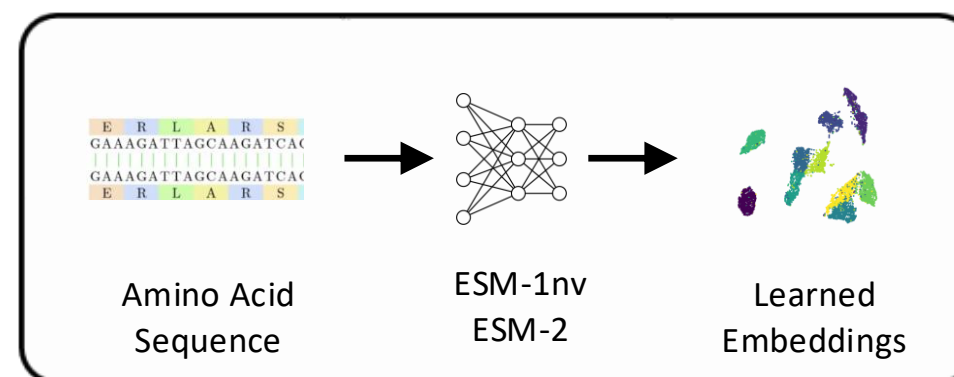
TRANSFORMING BIOLOGY / CHEMISTRY

9 Pre-trained Models Optimized for Drug Discovery – NVIDIA BioNeMo

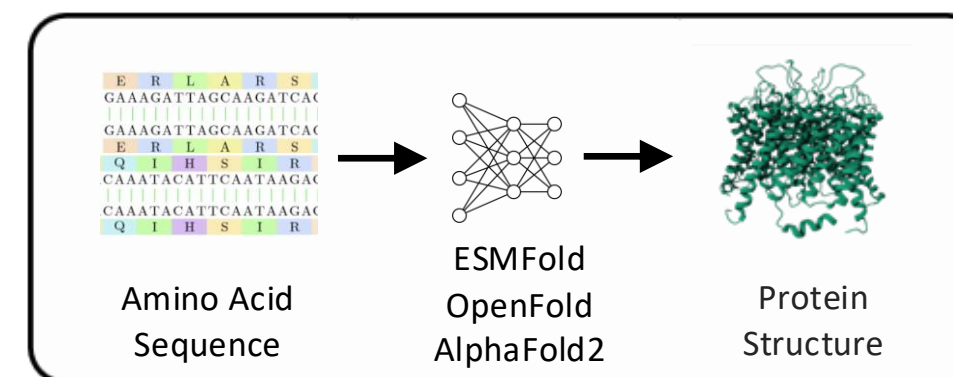
Protein Generation



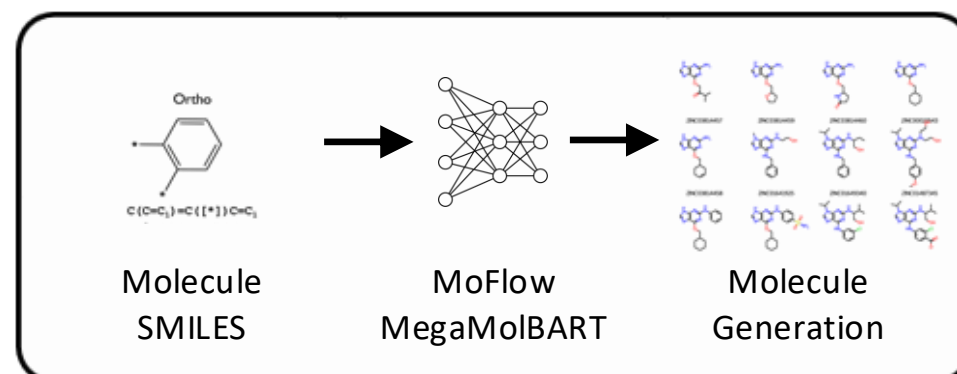
Protein Learned Sequence & Structure



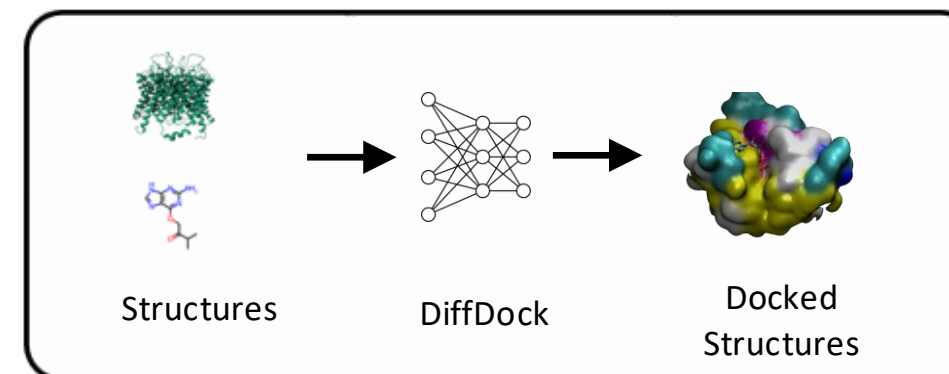
Protein Structure Prediction



Molecular Generation



Molecular Docking



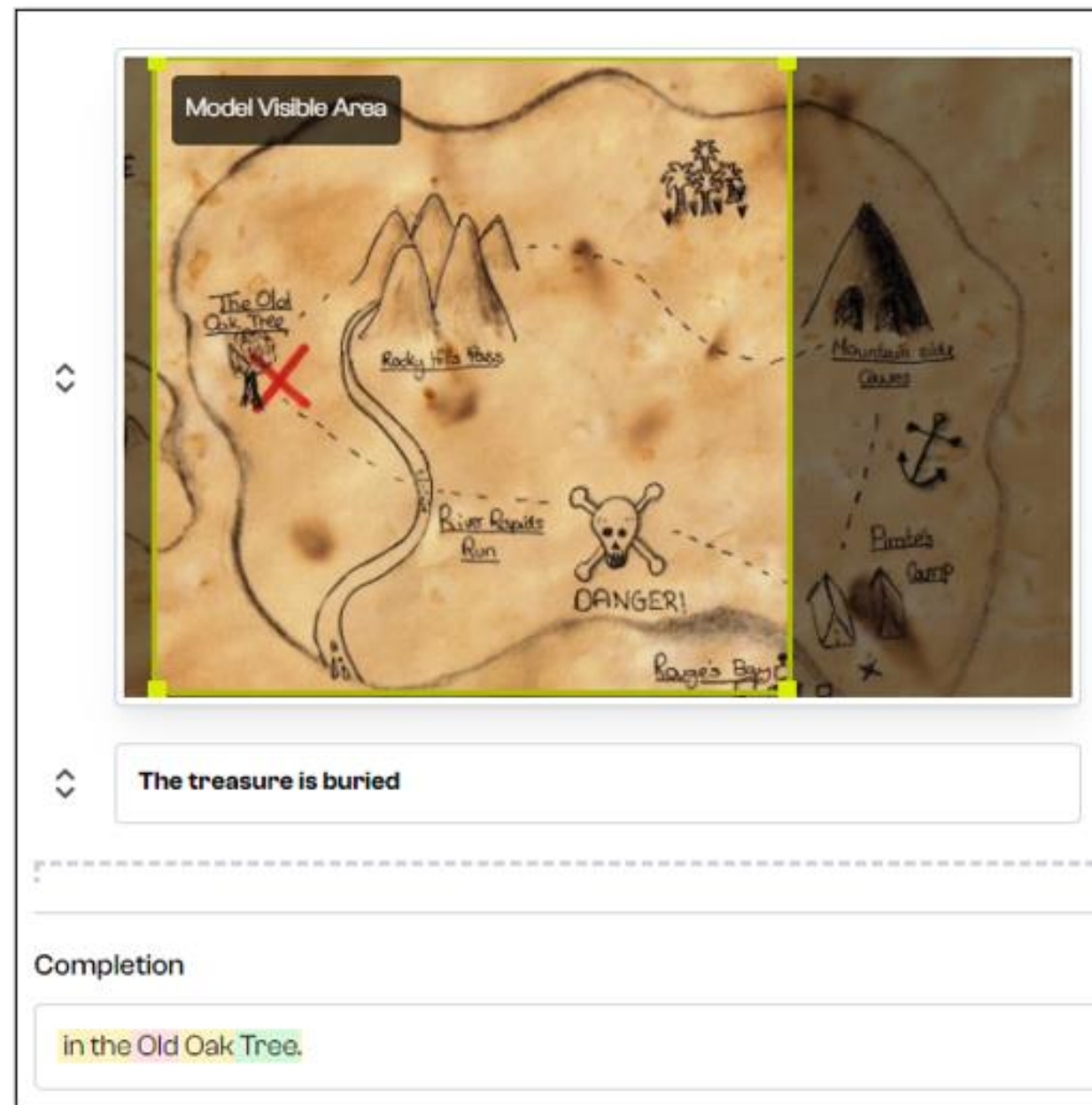
NVIDIA DGX Cloud

MULTIMODAL MODELS

The background features a smooth gradient from a vibrant green on the left to a deep blue on the right. Overlaid on this gradient is a complex, white, network-like pattern. This pattern consists of numerous small dots connected by thin, intersecting lines, creating a mesh or web-like structure that is denser on the right side of the image.

TRANSFORMING COMPUTER VISION

Vision Language Models



[Aleph-Alpha/magma](#)



What is wrong with the subject in the image? What can I do to fix it?

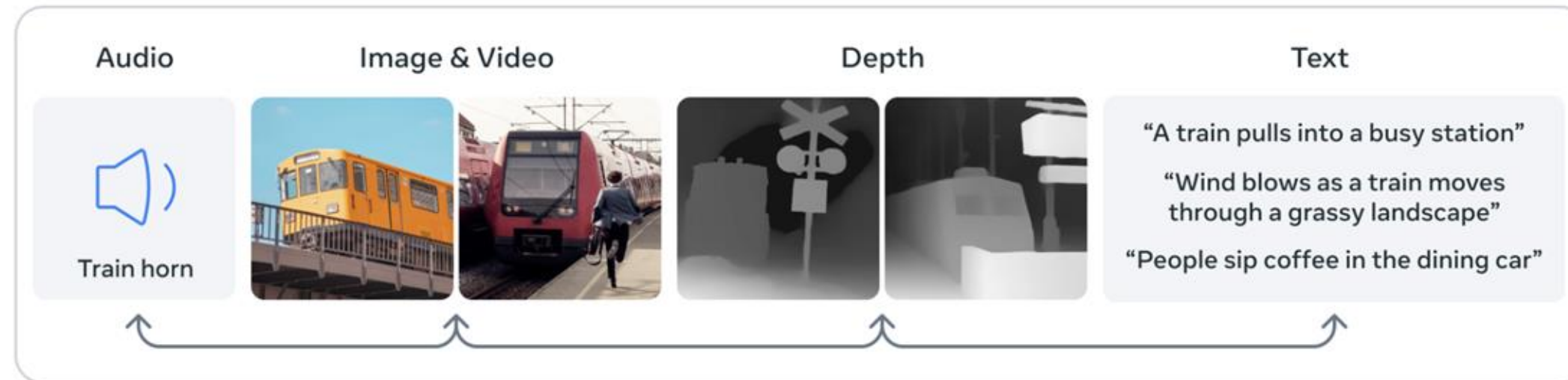
N The subject in the image is an old, run-down cabin that is covered in vines and surrounded by a lot of brush. This indicates that the cabin has been neglected for an extended period of time, and it has returned to a state of disrepair. To fix this situation, a thorough cleaning and restoration process would be required. This would involve removing the overgrown vines and brush, repairing any structural damage to the cabin, and repainting or restoring any visible wooden surfaces. Additionally, any remaining debris or garbage should be cleared away, and the interior of the cabin should be cleaned and inspected for any potential safety hazards. Once the restoration process is complete, the cabin can be used for its intended purpose once again or, if desired, repurposed according to the needs of the owner or the local community.

[NeVA 22B - Multi-modal vision-language model that understands text and images and generates informative responses.](#)

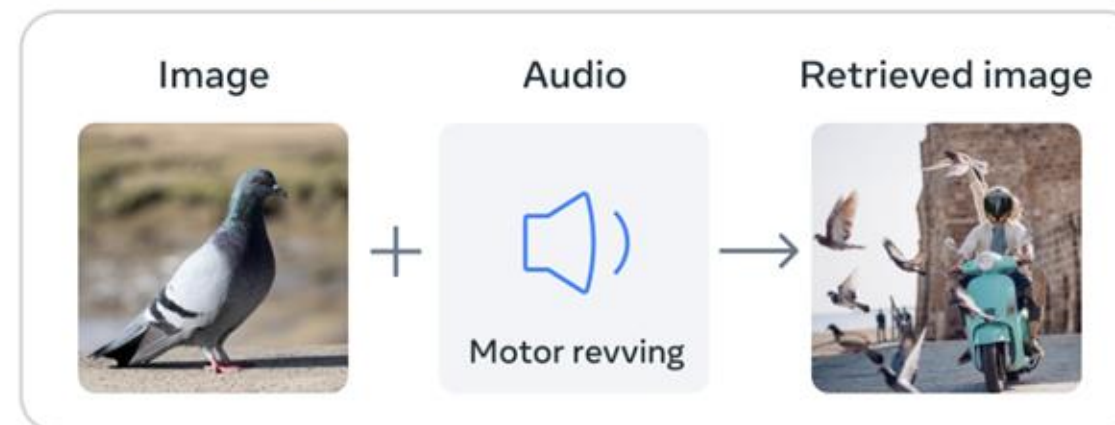
MULTI MODAL MODEL

Holistic AI learning across six modalities

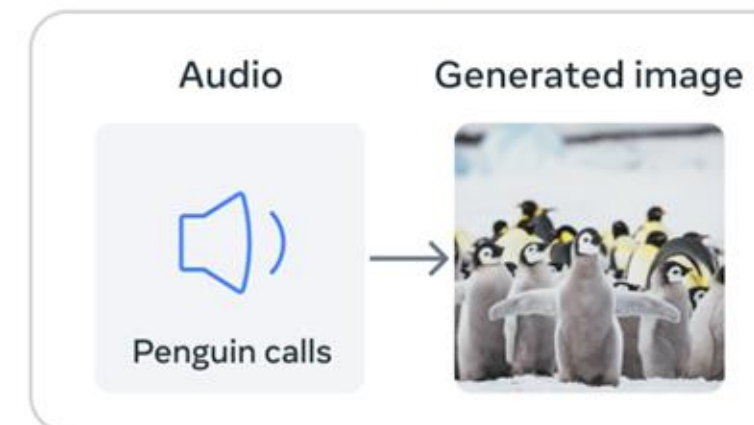
Cross-modal retrieval



Embedding-space arithmetic



Audio to image generation



By aligning six modalities' embedding into a common space, ImageBind enables cross-modal retrieval of different types of content that aren't observed together, the addition of embeddings from different modalities to naturally compose their semantics, and audio-to-image generation by using our audio embeddings with a pretrained DALLÉ-2 decoder to work with CLIP text embeddings.

EXAMPLE OF USE CASES FOR LARGE MODELS

USE CASES FOR LARGE MODELS



Create interactive avatars with NVIDIA Avatar Cloud Engine

[NVIDIA Omniverse ACE](#)



[NVIDIA Project Mellon](#)

Control your applications with natural language



[GPT-4 to Generate 3D Content in Omniverse](#)

Generate 3D content using natural language

PART 1

Motivation and basic concepts

- **Lecture**
 - Why large models?
 - Impact on AI landscape
 - **Challenges of large model training**
 - Basic techniques for memory reduction
 - Overview of the tools used in the lab
- **Lab 1 / Part 1**
 - Introduction to the SLURM class environment
 - GPT model pretraining
 - Multi-node scaling
 - Optimize the GPT model pretraining



SIZE

GOING BIGGER

Memory requirements during training need to account for more than model weights

Consider **1 billion parameters** model in **FP16** and do the math:

Data representation: Weights and Gradients in FP16

Adam optimizer: Store 12 bytes per weight in FP16

$$10^9 * (2B + 2B + 12B) = 14.90GB$$

1 billion parameters

2 bytes per weight

2 bytes per gradient

12 bytes per optimizer state



TIME

SCALE OF COMPUTE

Within reach of most companies

Model size	Attention heads	Hidden size	Number of layers	Number of parameters (billion)	Model-parallel size	Number of GPUs	Microbatch size	Batch size	Achieved teraFLOP/s per GPU	Percentage of theoretical peak FLOP/s	Achieved aggregate petaFLOP/s
1.7B	24	2304	24	1.7	1	32	16	512	137	44%	4.4
3.6B	32	3072	30	3.6	2	64	16	512	138	44%	8.8
7.5B	32	4096	36	7.5	4	128	16	512	142	46%	18.2
18B	48	6144	40	18.4	8	256	8	1024	135	43%	34.6
39B	64	8192	48	39.1	16	512	4	1536	138	44%	70.8
76B	80	10240	60	76.1	32	1024	2	1792	140	45%	143.8
145B	96	12288	80	145.6	64	1536	2	2304	148	47%	227.1
310B	128	16384	96	310.1	128	1920	1	2160	155	50%	297.4
530B	128	20480	105	529.6	280	2520	1	2520	163	52%	410.2
1T	160	25600	128	1008.0	512	3072	1	3072	163	52%	502.0

- ~6 weeks on 1 x DGX A100
- ~2 weeks on 4 x DGX A100

- ~65 weeks on 1 x DGX A100
- ~16 weeks on 4 x DGX A100

- ~5 years on 1 x DGX A100
- ~1 year on 4 x DGX A100

- ~69 years on 1 x DGX A100
- ~17 year on 4 x DGX A100

Weak scaling throughput for GPT models ranging from 1 billion to 1 trillion parameters.

Scaling up of training is a form of time compression enabling faster time to convergence



CHOOSE MODEL SIZE?

PREDICTABILITY OF LARGE MODELS

Optimal Model size given a compute budget

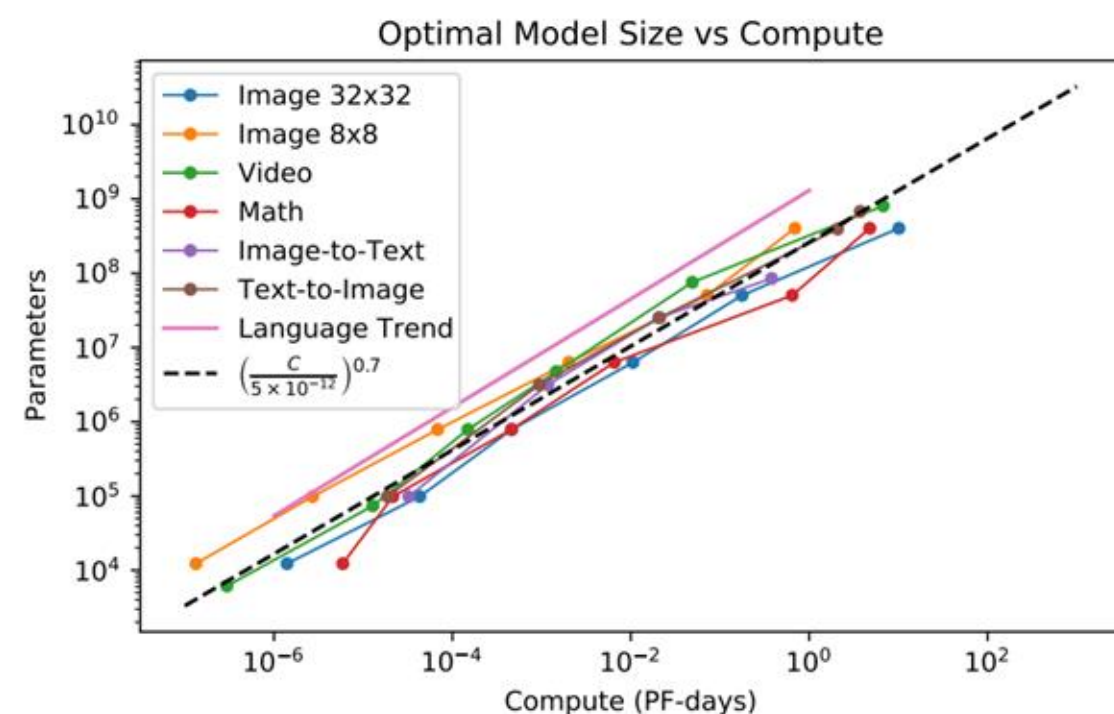


Figure 2 | Optimal model size is consistent across domains— We display the optimal model size N_{opt} as a function of the training compute budget C . Not only does $N_{\text{opt}}(C)$ behave as a power-law, but the behavior is remarkably similar for all data modalities.

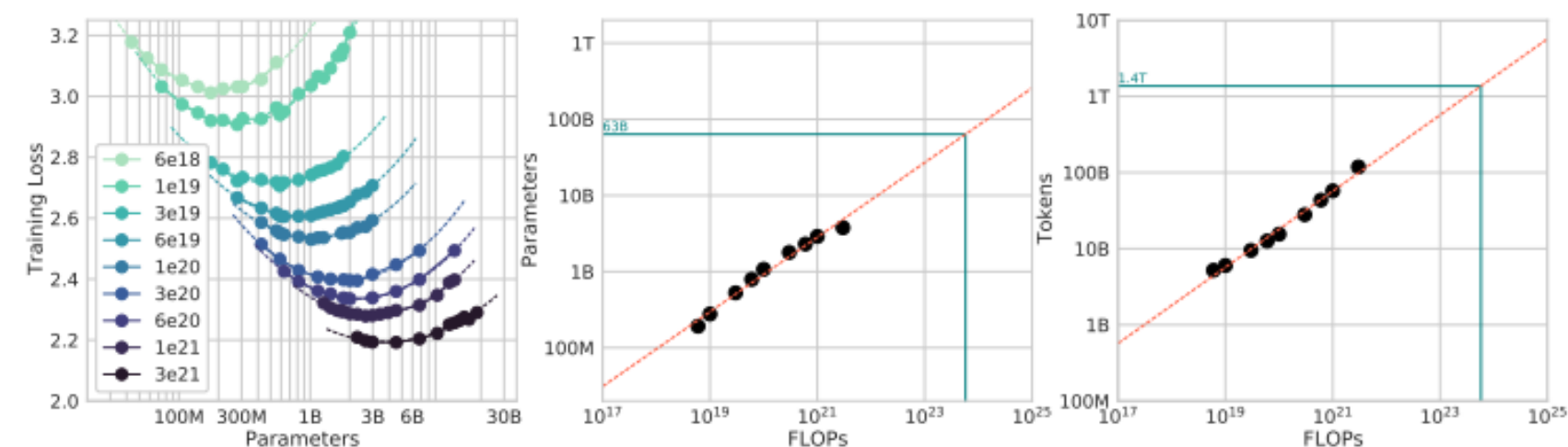


Figure 3 | IsoFLOP curves. For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

For a given compute budget you can estimate to optimal model size to train across different domains

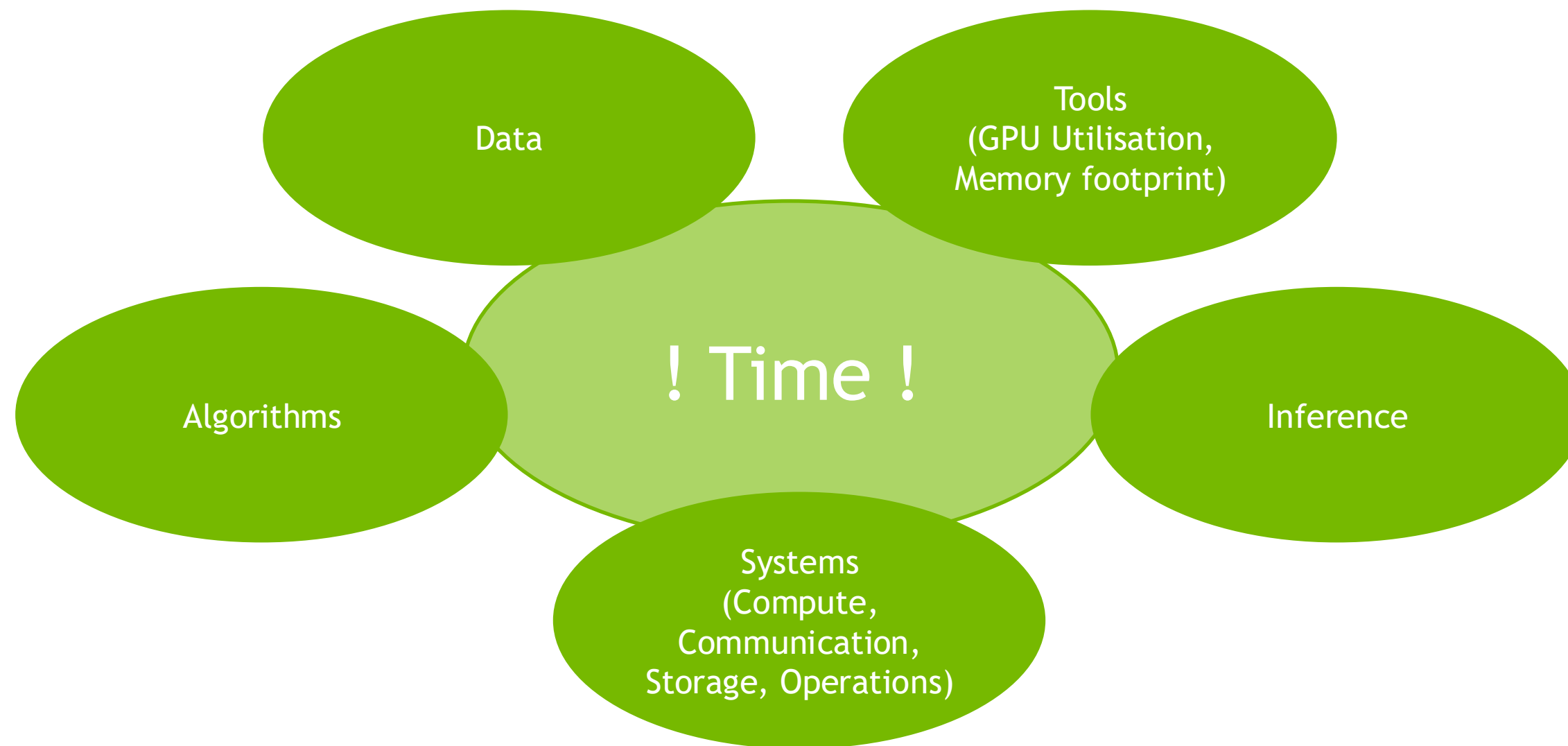
For a given compute budget you can estimate to optimal model size and number of tokens needed to achieve a specific loss value

THE CHALLENGES

The background of the slide features a smooth gradient from a deep green at the top to a bright white at the bottom. Overlaid on this gradient is a complex, abstract network of white dots and thin white lines, resembling a molecular structure or a digital network. The network is denser in the lower half of the image and fades into the white background at the bottom.

KEY CHALLENGES

Large models require large execution time - catastrophic impact of bottlenecks



PART 1

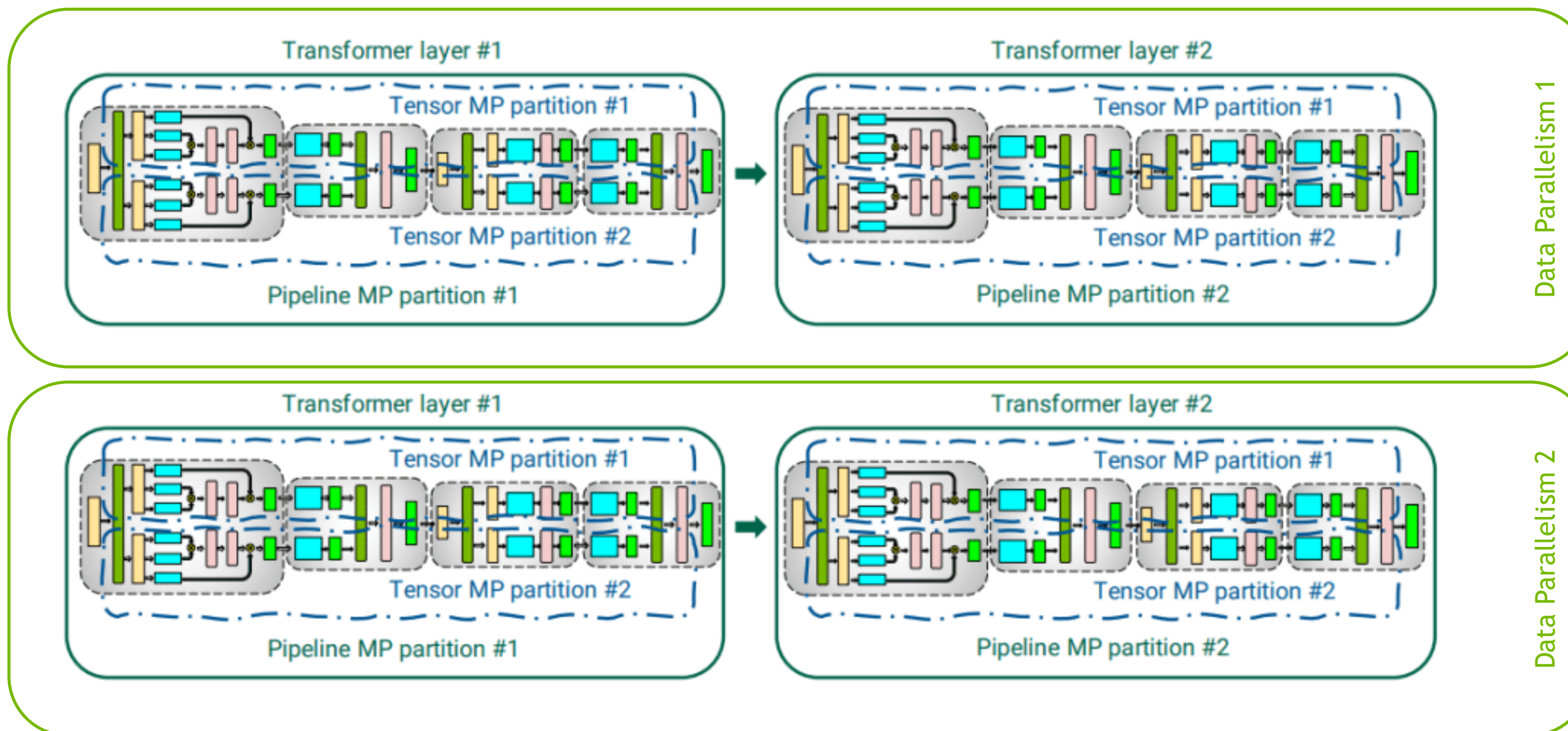
Motivation and basic concepts

- Lecture
 - Why large models?
 - Impact on AI landscape
 - Challenges of large model training
 - **Basic techniques for memory reduction**
 - Overview of the tools used in the lab

- Lab 1 / Part 1
 - Introduction to the SLURM class environment
 - GPT model pretraining
 - Multi-node scaling
 - Optimize the GPT model pretraining

DEALING WITH MEMORY CONSTRAINTS

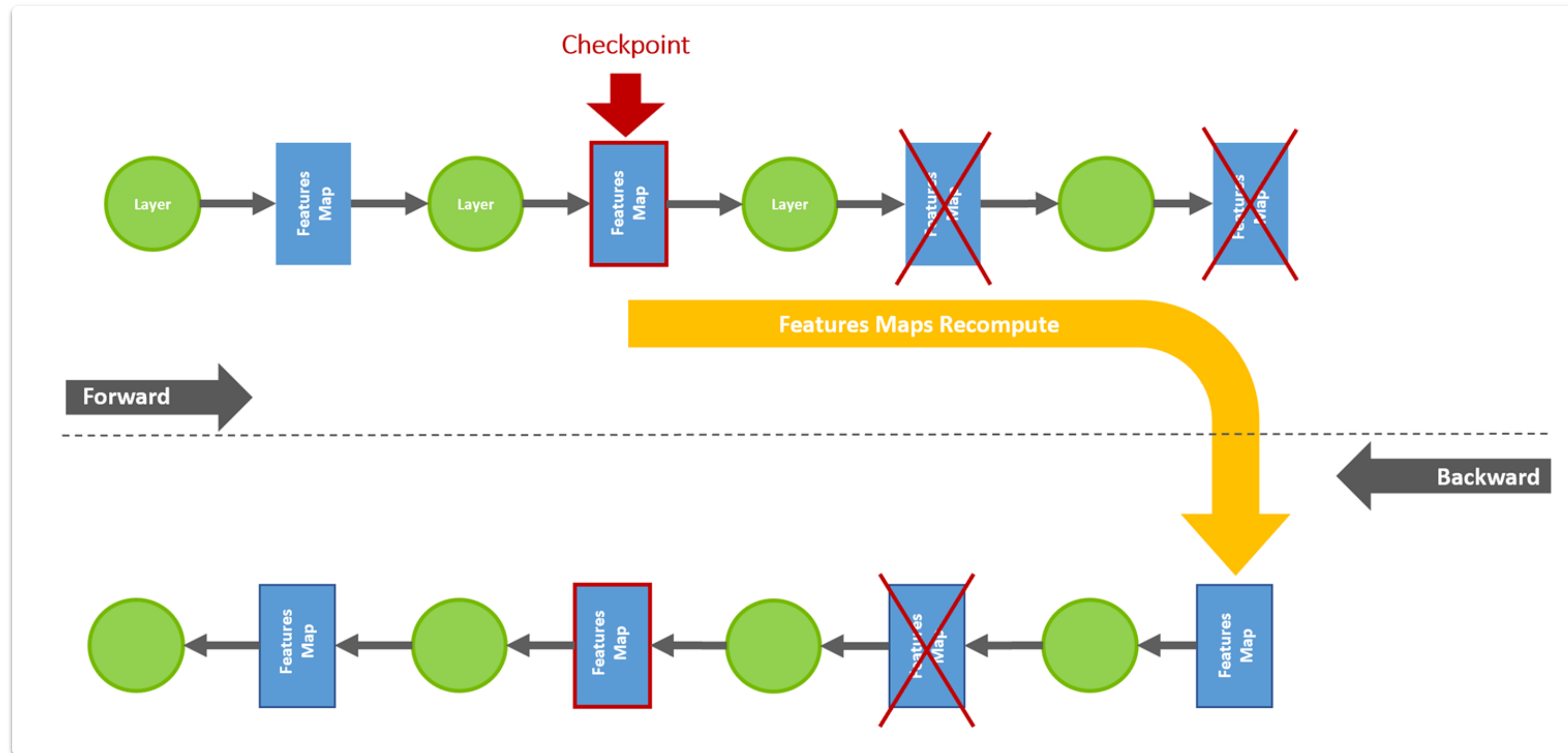
Various forms of parallelism - covered in the next part



Example of data, tensor and pipeline parallelism applied to the transformer architecture

DEALING WITH MEMORY CONSTRAINTS

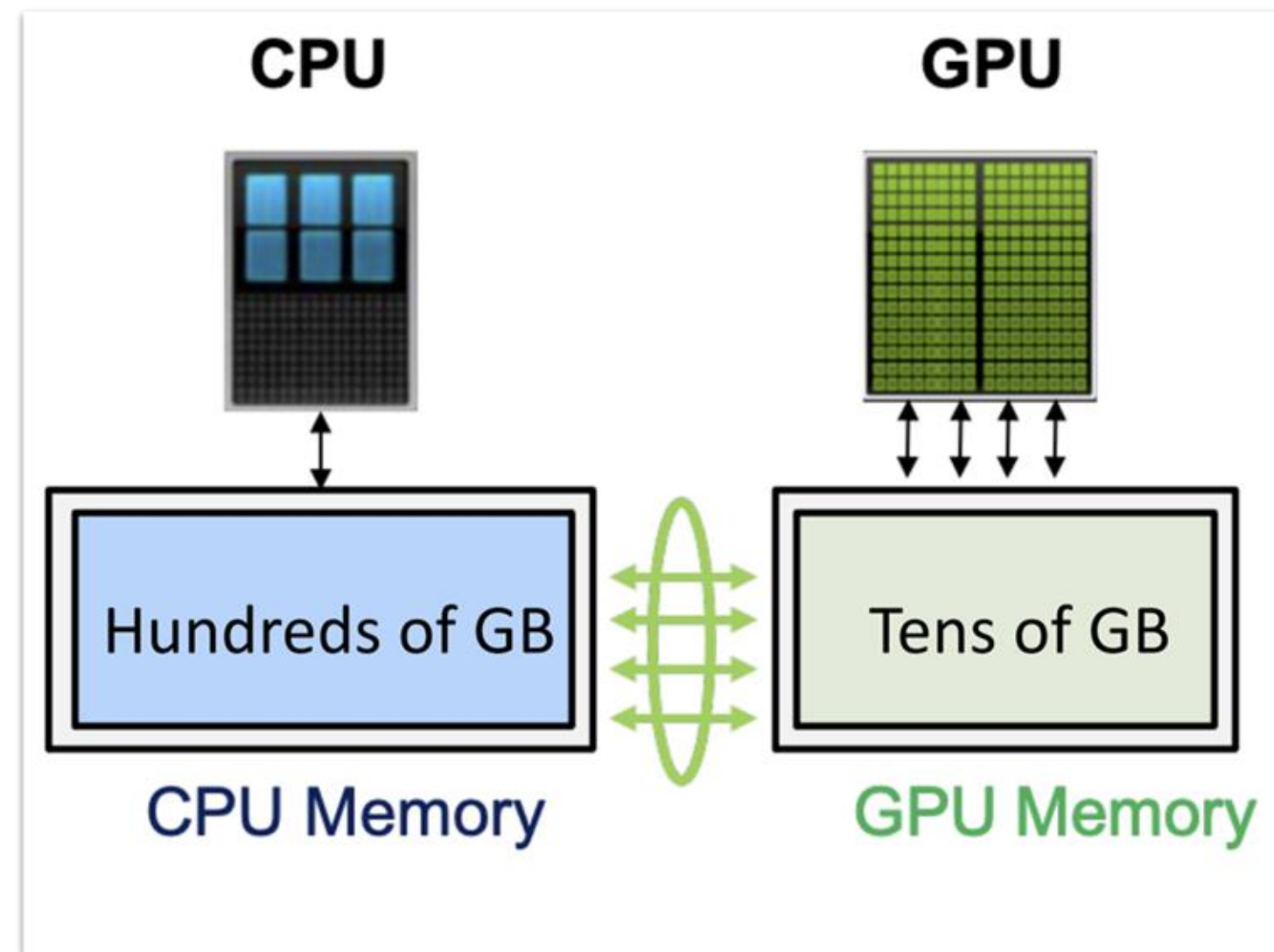
Activation Checkpointing



Instead of storing all activations (outputs inside the network), re-compute ones that are fast to compute to save memory

DEALING WITH MEMORY CONSTRAINTS

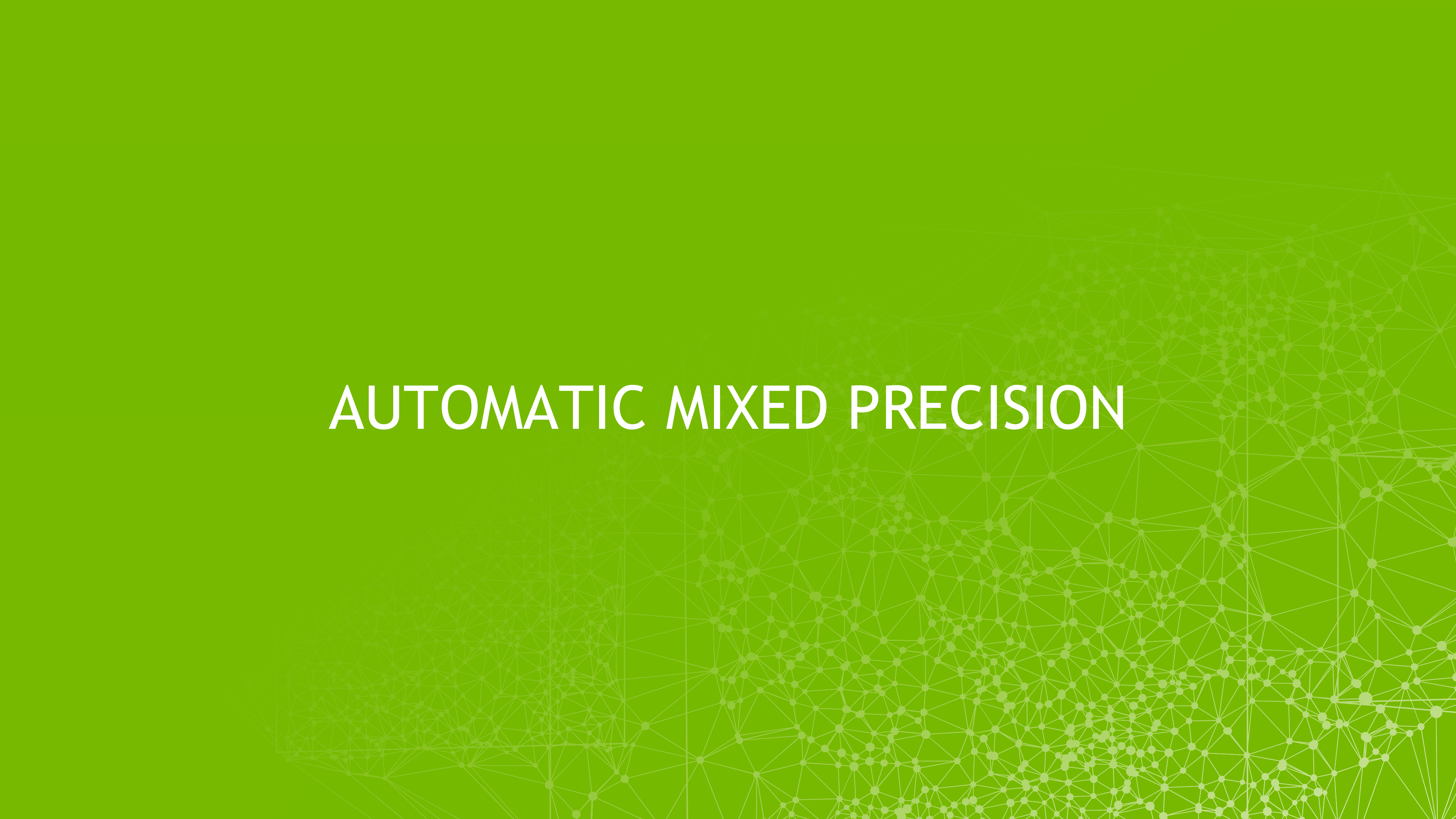
Tensor Offloading



Offload CPU tensors not used in computation from GPU to CPU

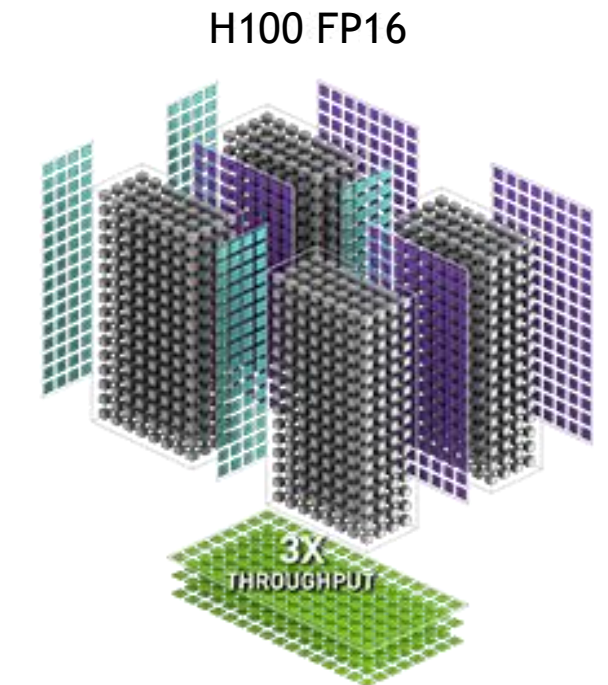
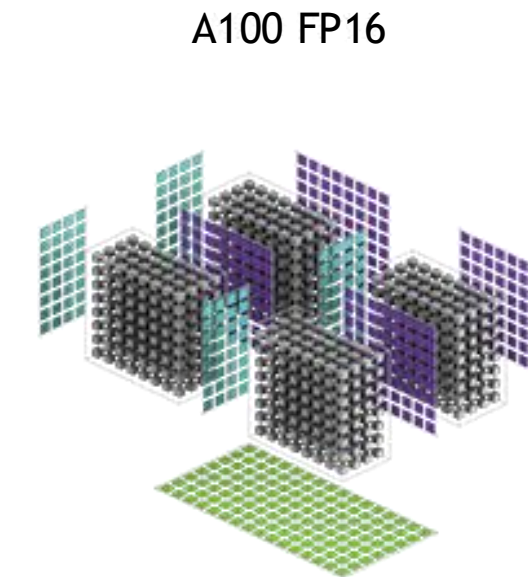
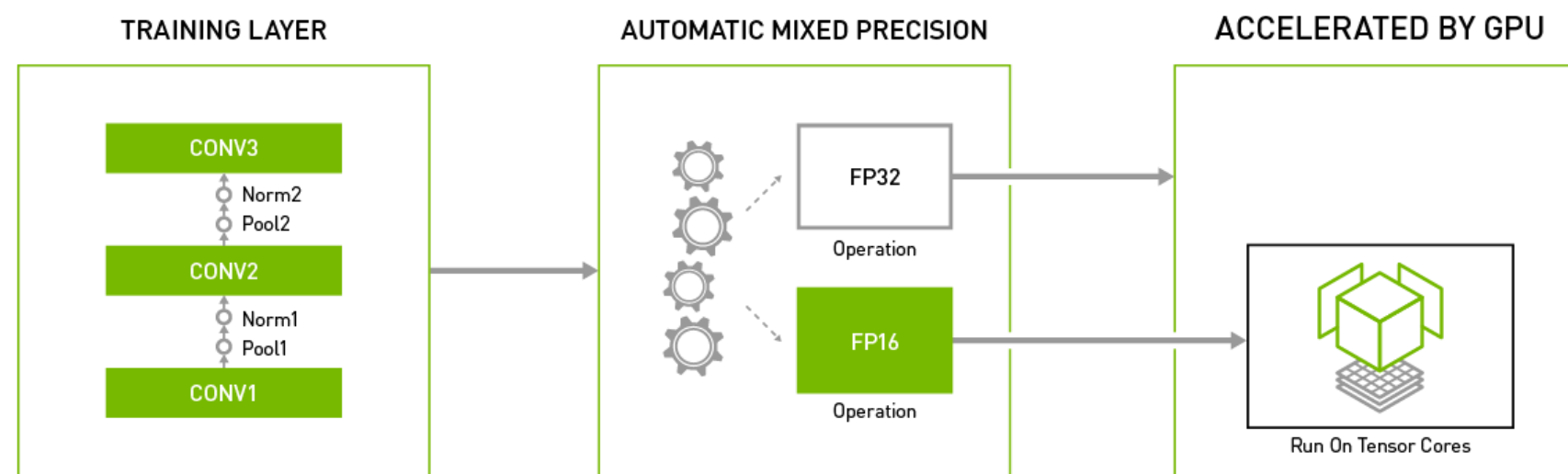
Training times will be slower due to slow data movement

AUTOMATIC MIXED PRECISION

The background features a smooth gradient from a deep green at the top to a bright yellow at the bottom. Overlaid on this is a complex, organic network of thin white lines connecting numerous small white dots, creating a mesh-like structure that resembles a molecular or neural network. The density of the network increases towards the bottom right corner.

DEALING WITH MEMORY CONSTRAINTS

Automatic Mixed Precision



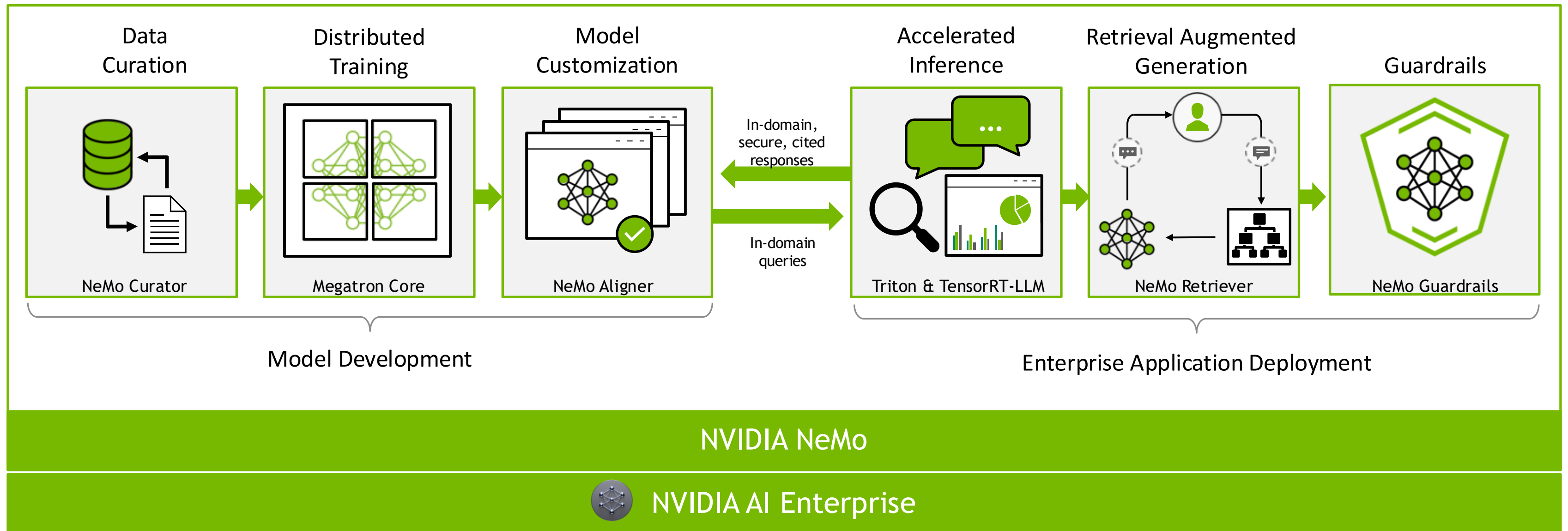
Training frameworks will train in different precisions for different layers of the model depending on the hardware being used

NEMO FRAMEWORK

The background of the slide features a smooth gradient from a deep blue at the top to a vibrant green at the bottom. Overlaid on this gradient is a complex, abstract network of thin white lines connecting numerous small white dots. This network pattern is most dense in the lower right quadrant and fades out towards the upper left, creating a sense of depth and connectivity.

NEMO FRAMEWORK

Build, customize and deploy generative AI models with NVIDIA NeMo



NVIDIA DGX Cloud



MODEL TRAINING AND DEPLOYMENT

NeMo gives organizations an easy, cost-effective, and fast way to adopt generative AI

End-to-End Pipeline



Complete solution across the LLM pipeline - data processing, training, and inference of generative AI models.

Accelerated Performance



Multi-node and multi-GPU training and inference maximize throughput and minimize LLM training time.

Production Ready



Secure, optimized, full-stack for production deployments with support, security, and API stability.

Run Anywhere



Train and deploy generative AI anywhere - all major clouds, data center, or at the edge.

Increased ROI



Quickly train, customize, and deploy LLMs at scale, reducing time to solution and increasing ROI.

PART 1

Motivation and basic concepts

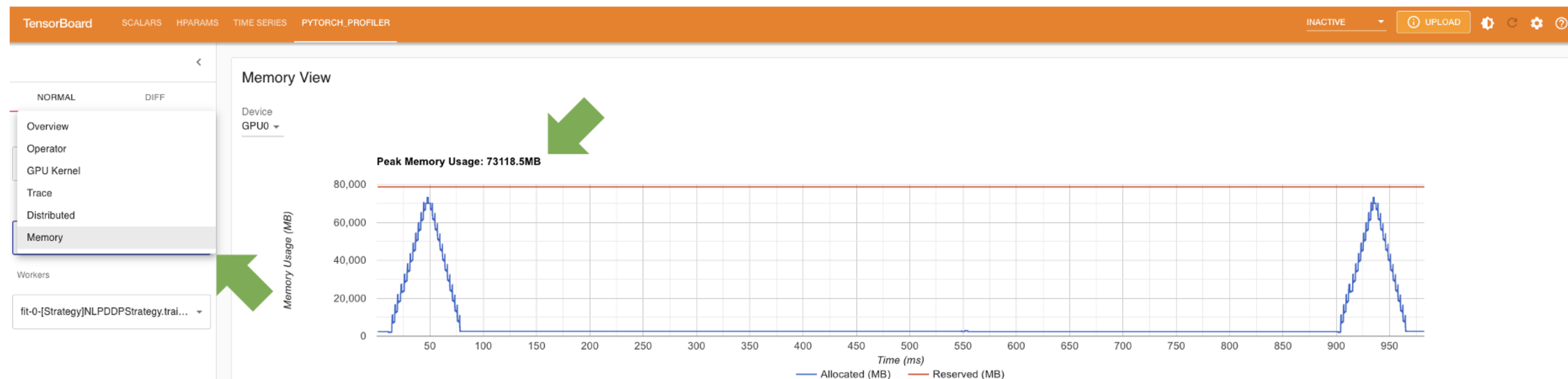
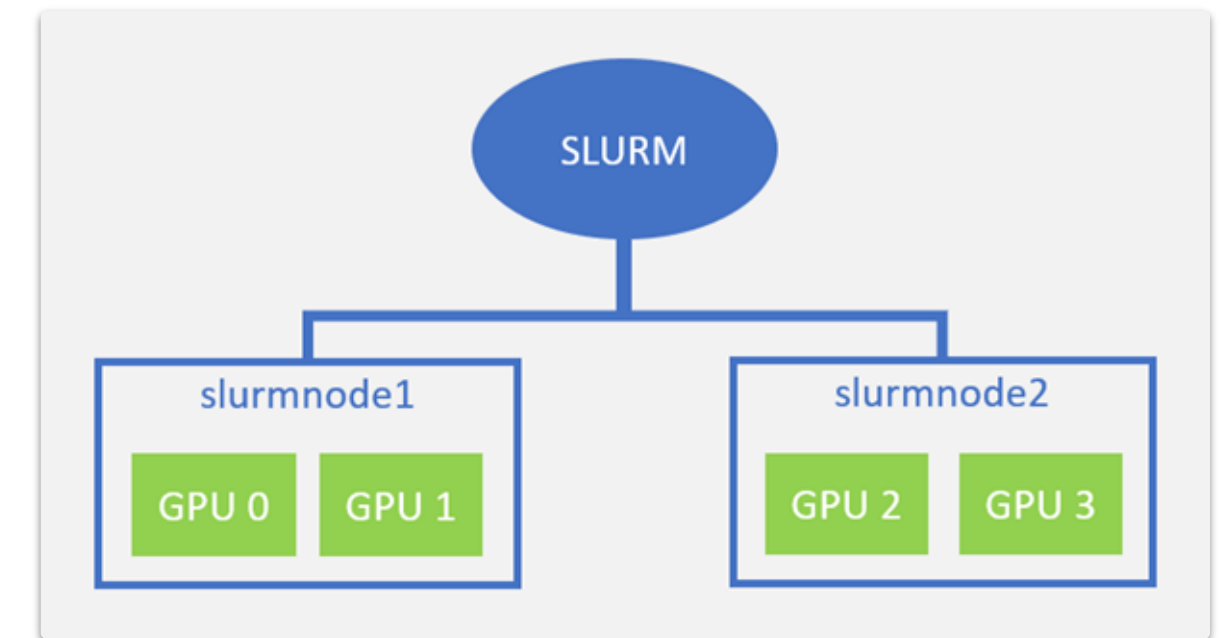
- **Lecture**
 - Why large models?
 - Impact on AI landscape
 - Challenges of large model training
 - Basic techniques for memory reduction
 - **Overview of the tools used in the lab**

- **Lab 1 / Part 1**
 - Introduction to the SLURM class environment
 - GPT model pretraining
 - Multi-node scaling
 - Optimize the GPT model pretraining

THE LAB 1 / PART 1

Overview

- Introduction to the SLURM class environment
- Multi-GPU and Multi-node GPT pretraining with NVIDIA NeMo
- Profile with *Pytorch_Profiler* and optimize the GPT model pretraining

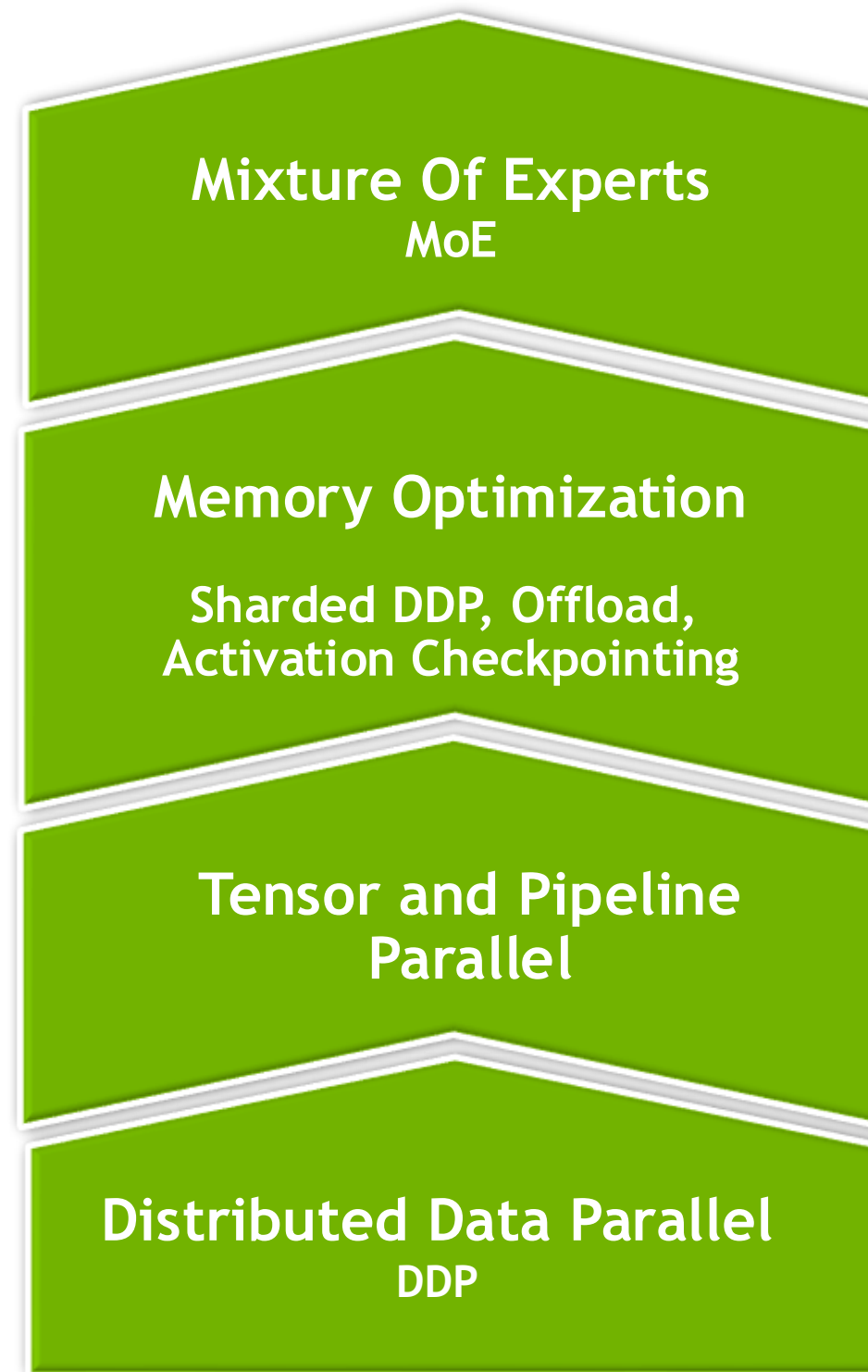




IN THE NEXT CLASS

NEXT CLASS

Discuss advanced concepts for optimized large scale distributed training



PART 1

Motivation and basic concepts

- Lecture
 - Why large models?
 - Impact on AI landscape
 - Challenges of large model training
 - Basic techniques for memory reduction
 - Overview of the tools used in the lab

- Lab 1 / Part 1
 - Introduction to the SLURM class environment
 - GPT model pretraining
 - Multi-node scaling
 - Optimize the GPT model pretraining